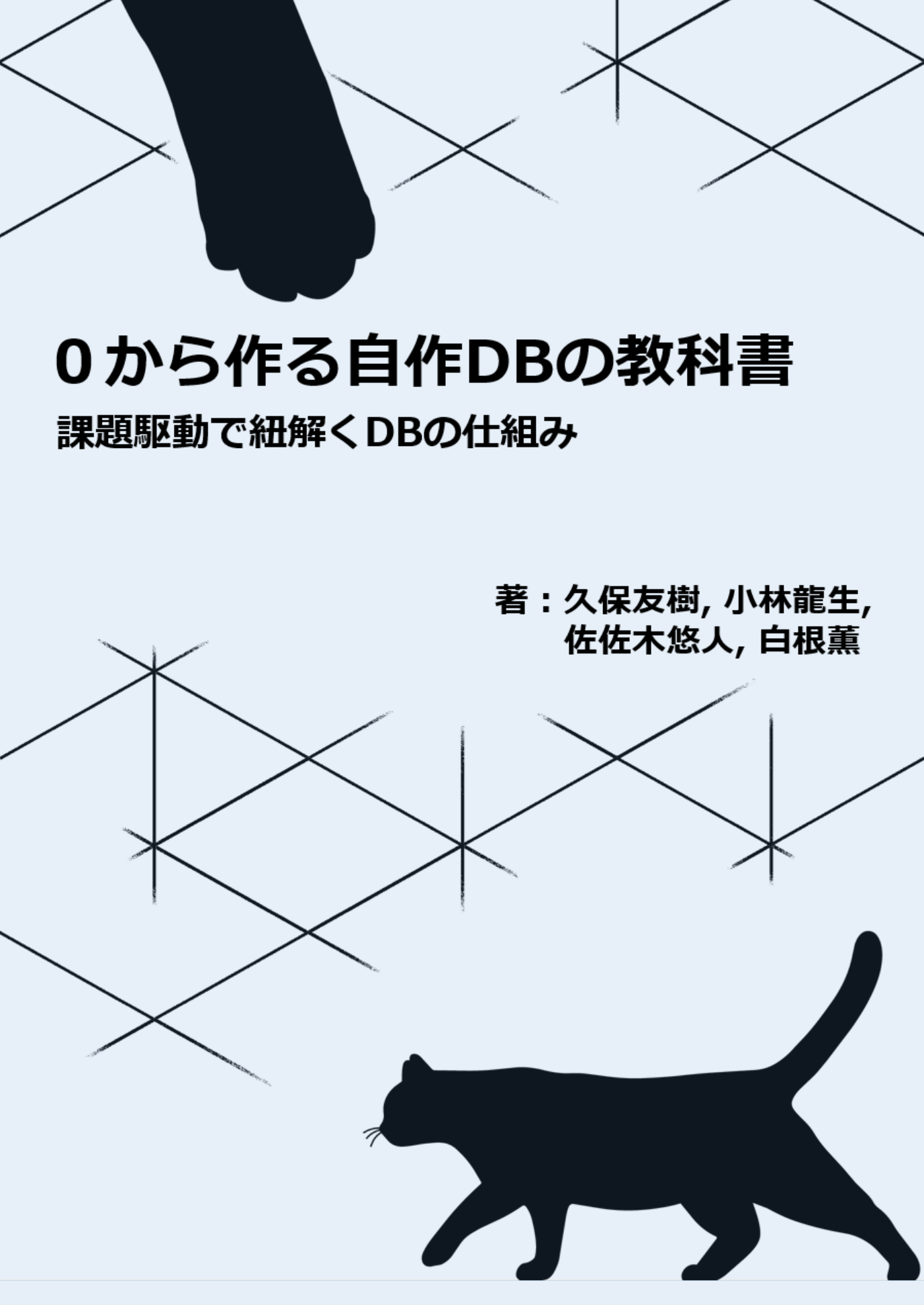




0から作る自作DBの教科書

課題駆動で紐解くDBの仕組み

著：久保友樹, 小林龍生,
佐佐木悠人, 白根薫

0 から作る自作 DB 入門

久保 友樹、小林 龍生、佐佐木 悠人、\白根
薫 著

2026-07-12 版 くろねこチーム 発行

まえがき

現代のソフトウェア開発において、データベースはなくてはならない存在です。Webアプリケーション、スマートフォンアプリ、企業の基幹システムに至るまで、ありとあらゆるシステムがデータベースを利用しています。

しかし、アプリケーション開発者にとって、その内部はしばしば「ブラックボックス」になりがちです。SQL という呪文（クエリ）を投げると、データベースは裏側で良きに計らい、魔法のように高速にデータを返してくれます。「なぜインデックスを張ると速くなるのか」「内部でデータはどのような形でディスクに保存されているのか」を深く理解する機会は多くありません。

既存の学習手法の限界と本書のアプローチ

データベースの仕組みを学ぼうとして、既存の技術書を手に取ったことがあるかもしれません。しかし、従来のデータベース学習には以下のような壁がありました。

- **理論が先行して難しすぎる:** B-Tree やリレーショナル代数などの学術的な理論ばかりが続き、コードを書く前に挫折してしまう。
- **完成形しか見えない:** 「データベースとはこういうものだ」と最初から複雑なコードを見せられ、「なぜその設計が必要になったのか」という背景が分からない。
- **コードが巨大すぎる:** PostgreSQL や MySQL などの実際のソースコードを読もうとしても、数百万行の海に溺れてしまう。

そこで本書では、既存のアプローチとは全く異なる手法をとります。私たちが目指するのは、最初から完璧なデータベースを作ることではありません。「**進化の過程（変化）**」を楽しむこと。これこそが、本書の最大のテーマです。

「**課題駆動型（Problem-Driven）**」で、最小のコードから少しずつデータベースを育てていくという体験を通して、技術がなぜ必要なのかを肌で感じてください。「データが増えると遅い」「更新のたびにファイル全体を書き直さなければならない」といった課題にぶつかり、先人たちが生み出したデータ構造やアルゴリズムを実装し、劇的にパフォー

マンスが向上する快感を味わう。その一つひとつの試行錯誤の中にこそ、データベースの真髓が隠されています。

サンプルコードについて

本書で実装するデータベース「nekoDB」の全ソースコードは、以下の GitHub リポジトリで公開しています。各章のコードを直接確認したり、手元で動かしたりしながら読み進めてください。

<https://github.com/Yutosaki/team-kuroneko/tree/master/src>

対象読者

本書は、以下のような方々に向けて書かれています。

- 普段データベースを使っているが、裏側の仕組みを知りたい開発者: ORM や SQL を使ってアプリケーションを作れるが、データベース内部で何が起きているのかはよく分からないという方。
- 理論だけでなく、コードを書いてシステムを理解したいエンジニア: 「B-Tree」や「実行計画 (プランナ)」といった言葉や概念は知っているが、それをどうやって実装するのか具体的なイメージが湧かない方。
- 低レイヤやミドルウェアの開発に興味がある方: 自作コンパイラや自作 OS のように、OS のファイルシステムとアプリケーションの間を取り持つ「ミドルウェア」を自分の手で構築してみたい方。

本書の進め方と各章の流れ

本書のすべての章は、「最小限の実装を作る」→「データが増えるなどの壁 (課題) にぶつかる」→「解決策となる理論を学ぶ」→「実装して改善する」というサイクルで進みます。具体的には以下のロードマップで本格的なデータベースを作り上げていきます。

- 第 1 章 データベースとは何か: データベースの役割や、単なるファイルとの違い、そして全体的なアーキテクチャについて学びます。
- 第 2 章 最小のデータベース: 対話型のプログラム (REPL) を作り、メモリ上だけで動く最小のデータベースを作ります。「電源を切るとデータが消える」という課題に直面します。
- 第 3 章 データの永続化: データをファイルに書き込んで永続化します。しかし、

「更新のたびにファイル全体を書き直さなければならない」という非効率さに気づきます。

- **第4章 ページ (Page) によるデータ配置:** 「ページ」という概念を導入し、ディスクの読み書きを効率化します。しかしデータが増えると「検索が遅すぎる (線形探索)」問題に直面します。
- **第5章 B-Tree による検索の高速化:** 検索を劇的に高速化するデータ構造「B-Tree」を導入します。ここで「独自のコマンドでは操作が複雑すぎる」というインターフェースの課題が出てきます。
- **第6章 テーブル定義と SQL の実行:** 人間が直感的に操作できるように、SQL の構文解析 (パーサ) を実装します。ついにデータベースらしい姿になりますが、ここではまだ「せっかく作った B-Tree がクエリから利用されず、常に全件走査 (フルスキャン) されてしまう」という実行効率の壁にぶつかります。
- **第7章 インデックスの適用と実行計画:** クエリの条件を見て「インデックスを使うべきか、全件走査すべきか」を選択する、実行計画 (プランナ) の基礎を実装します。無事に SQL からの高速検索を実現しますが、今度は「ストレージ処理から実行エンジンまでが密結合し、コードが複雑に絡み合ってしまう」という設計・保守上の課題が浮き彫りになります。
- **第8章 構造の分離:** その複雑化の壁を越えるため、システムを適切な層 (レイヤー) に整理し直すりファクタリングを行います。各コンポーネントの責任境界を明確にし、拡張性の高いアーキテクチャを完成させます。
- **第9章 真のデータベースに向けて:** これまでのアーキテクチャを振り返り、本物の商用データベースとの違いや、今後の学習の道標を示します。

本書を読み終える頃には、単なる文字列である SQL がどのように解析され、実行計画が練られ、B-Tree のインデックスを辿り、ディスク上のページからデータが読み取られるのか——その壮大なパケツリレーの全貌を、自分の書いたコードを通して説明できるようになるでしょう。

それでは、いよいよ私たちだけのデータベースを作り始めましょう。

目次

まえがき	2
既存の学習手法の限界と本書のアプローチ	2
サンプルコードについて	3
対象読者	3
本書の進め方と各章の流れ	3
第 1 章 データベースとは何か	8
1.1 データベースの役割	8
1.2 ファイルとの違い	9
1.3 データベースの内部アーキテクチャ	10
第 2 章 最小のデータベース	12
2.1 本章の方針：メモリ上で動く最小構成	12
2.2 対話型プログラム（REPL）の作成	13
2.3 入力とコマンドの扱い	14
2.4 CRUD 操作の実装	16
2.5 ハッシュテーブルによるオンメモリ管理	19
2.6 まとめと次章への課題	20
第 3 章 データの永続化	22
3.1 第 2 章までの課題：揮発的なデータ保持	22
3.2 本章の方針：テキストファイルへのレコードの書き出し	22
3.3 ファイルを扱う準備	23
3.4 データの直列化（シリアライズ）設計	24
3.5 ファイルへの保存	25
3.6 永続化の確認	27
3.7 まとめと次章への課題	28

第 4 章	ページとスロットによるデータ配置の改善	31
4.1	第 3 章までの課題：ファイル全体の書き直し	31
4.2	本章の方針：ページ単位での読み書き	31
4.3	ページの導入と固定長スロットによるレコードの設計	32
4.4	ページ単位での読み書きの実装	34
4.5	処理時間の計測とパフォーマンス評価	38
4.6	まとめと次章への課題	39
第 5 章	B-Tree による検索の高速化と領域管理	40
5.1	第 4 章までの課題：線形探索の限界	40
5.2	本章の方針：インデックスによる直接アクセス	40
5.3	検索を高速化するデータ構造の導入	41
5.4	B-Tree の採用：RecordId の導入	42
5.5	B-Tree の実装	43
5.6	起動時の B-Tree 構築と空き領域管理	48
5.7	B-Tree を用いた CRUD の高速化	49
5.8	まとめと次章への課題	51
第 6 章	テーブル定義と SQL の実行	52
6.1	第 5 章までの課題：独自コマンドの限界とスキーマの欠如	52
6.2	本章の方針：SQL と任意のテーブル定義	52
6.3	リレーショナルモデルと SQL 処理の導入	53
6.4	テーブル定義とカタログの設計	55
6.5	字句解析 (Tokenizer) の実装	59
6.6	構文解析 (Parser) と抽象構文木 (AST) の構築	62
6.7	クエリエグゼキュータによる AST の実行	68
6.8	実行結果のフォーマットと表示	72
6.9	まとめと次章への課題	74
第 7 章	プランナによる実行計画の選択	76
7.1	第 6 章までの課題：常に全件走査する SELECT	76
7.2	本章の方針：最適な実行経路の選択	76
7.3	実行計画とプランナの導入	77
7.4	カラムへのインデックス定義とカタログ保存	78
7.5	インデックスの構築と同期処理	79
7.6	プランナによる実行方法 (経路) の選択	80

7.7	計画フェーズと実行フェーズの分離	80
7.8	実行例とプランの確認	81
7.9	まとめと次章への課題	82
第 8 章	オペレータによる実行処理の分離	83
8.1	第 7 章までの課題：実行処理とストレージの密結合、メモリ枯渇	83
8.2	本章の方針：実行処理の抽象化とパイプライン化	83
8.3	オペレータとイテレータ（Volcano）モデルの導入	84
8.4	走査オペレータの実装	86
8.5	行を加工するオペレータの実装	87
8.6	更新系オペレータの実装	89
8.7	プランナによる実行計画ツリーの構築	91
8.8	クエリエグゼキュータによるオペレータの実行	92
8.9	実行結果の確認	93
8.10	まとめと次章への課題	94
第 9 章	真のデータベースに向けて	96
9.1	これまでの振り返り	96
9.2	今に足りないものと実装方針	97
9.3	おわりに：ブラックボックスを開けよう	98
あとがき		100
著者紹介		101

第 1 章

データベースとは何か

いよいよデータベースを作り始めるにあたり、本章では「そもそもデータベースとはどのような役割を持っているのか」、そして「単なるファイルにデータを保存することと何が違うのか」を整理します。あわせて、これから私たちが構築していくデータベースの全体的なアーキテクチャについても俯瞰してみましょう。

1.1 データベースの役割

現代のソフトウェア開発において、データベースはなくてはならない存在です。Web アプリケーション、スマートフォンアプリ、企業の基幹システムに至るまで、ありとあらゆるシステムがデータベースを利用しています。

データベースが担う役割

データベースの最も根源的な役割は、「**データを安全に保管し、必要なときに高速に取り出せるようにすること**」です。アプリケーションの裏側には常にデータベースが鎮座しており、ユーザーのプロフィール、商品の在庫、日々の決済履歴など、システムにとって最も価値のある「状態」を記憶し続けています。

データを管理するだけでは十分ではない

しかし、ただ単にデータを「保存する」だけであれば、データベースという大掛かりな仕組みは必要ないように思えます。実際のシステム運用においては、データが増え続ける中で「数千万件のデータから、特定の 1 件を 1 ミリ秒で探し出す」といった圧倒的なパフォーマンスが求められます。さらに、「様々な条件を組み合わせで柔軟にデータを抽出する」「データを規則正しい構造（テーブル）として矛盾なく管理する」といった、高度なデータ処理能力が必要になります。

データベースが提供する機能

これらの厳しい要求に応えるため、リレーショナルデータベース（RDBMS）は以下のような強力な機能を提供しています。

- **検索の高速化（インデックス）**：膨大なデータの中から、目的のデータを一瞬で探し出す機能。
- **効率的なデータ配置**：ディスクへの読み書き（I/O）回数を極限まで減らし、大量のデータを高速に処理する機能。
- **構造化されたデータ管理**：データを「テーブル」と「カラム」という形式で整理し、型（数値や文字列など）を厳格に管理する機能。
- **柔軟な問い合わせ（SQL）**：「20代で、かつ先月商品を購入したユーザー」のような複雑な条件を、人間が直感的に記述できるインターフェース。

私たちが普段何気なく使っているデータベースは、単なるデータの入れ物ではなく、これらの「高速性」と「利便性」を高度なアルゴリズムによって両立させているのです。

1.2 ファイルとの違い

データベースについて学ぶ際、最初にぶつかる疑問があります。「データを保存するだけなら、OS が提供している『ファイル（テキストや CSV）』に書き込めば十分ではないか？」という疑問です。

ファイルは「保存場所」である

確かに、数件から数千件程度の個人的なデータであれば、プログラムから直接ファイルを開いて追記・読み込みを行うだけで事足ります。ファイルシステムは、OS が提供する立派な「データの保存場所」です。

データベースはデータを管理する

しかし、「単なるファイル」はデータを物理的に配置する場所を提供しているだけで、その中身の整理や検索効率の面倒までは見てくれません。ファイルの中身をどう読み解き、どう検索するかは、すべてアプリケーション側（プログラマ）の責任になります。

データ量が増えると違いはさらに大きくなる

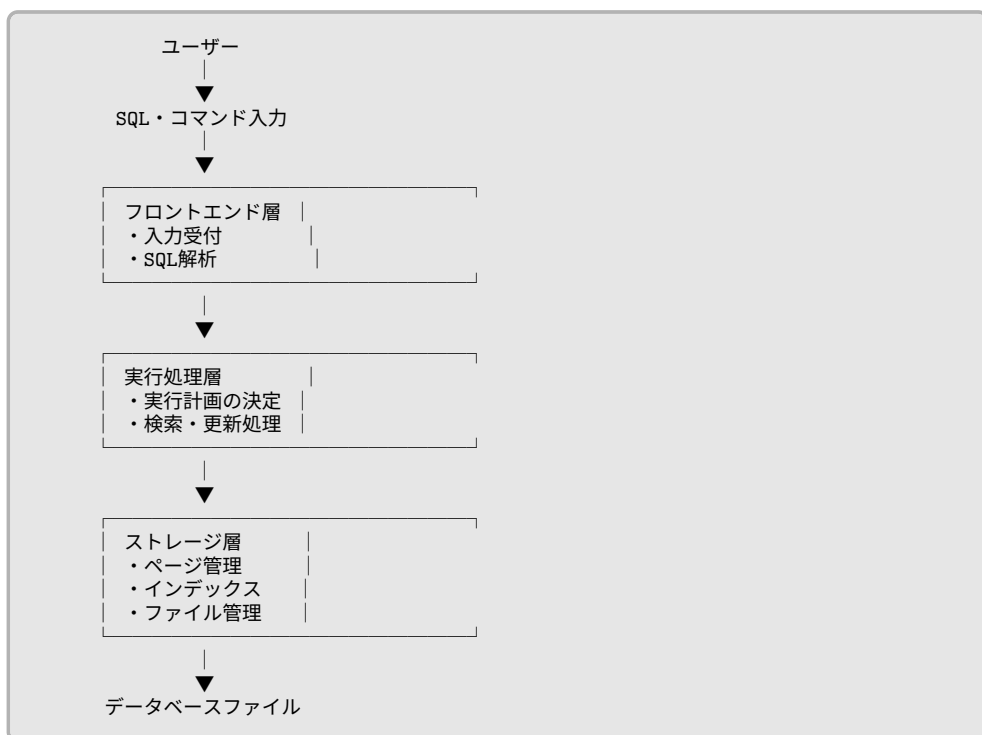
データ量が数百万件規模になり、要件が複雑化し始めると、単純なファイルベースの管理では以下のような致命的な問題が噴出します。

- **検索が遅い:** ファイルの後半にあるデータを探すために、毎回ファイルの先頭からすべてを読み込んで確認しなければなりません（線形探索）。
- **更新が非効率:** ファイルの途中にある数文字のデータを書き換えるだけでも、最悪の場合ファイル全体を再作成しなければならないことがあります。
- **複雑な条件で探せない:** ファイルから「年齢が20歳以上で、名前に'A'が含まれる人」を探すには、そのためのプログラムを毎回イチから書かなければなりません。

データベースシステム（DBMS）とは、言わば「生身のファイルシステムとアプリケーションの間に入り、データの配置から検索処理までを極限まで最適化してくれる仲介役」なのです。

1.3 データベースの内部アーキテクチャ

私たちがこれから本書で作っていくデータベースは、最終的に大きく分けて3つの役割（層）で構成されるアーキテクチャへと進化します。



各層が独立して役割を分担することで、機能を追加しやすく、保守しやすい構成になり

ます。本書では最初からこのすべてを作るわけではありません。次章の「第 2 章」では、まずはこの図の中の「入力受付」とメモリ上での単純な「検索・更新」だけを行う、非常に小さなプログラムから開発をスタートさせます。

第2章

最小のデータベース

前章では、データベースを段階的に作り上げていくロードマップを示しました。本章では、その第一段階として、まずは対話型のプログラム（REPL）を作り、メモリ上だけで動く最小のデータベースを作ります。

2.1 本章の方針：メモリ上で動く最小構成

本章では、まだディスク（ファイル）には触れず、まずはプログラムの実行中のみデータを保持する「オンメモリ」のデータベースを作成します。本格的なアーキテクチャを組む前に、データベースの「外枠」となる対話型のインターフェースと、最も基本的なデータ操作の仕組みを完成させることが目的です。

具体的には、以下の順序で実装を進めます。

1. **REPL の作成:** ユーザーからの入力を無限ループで受け付け、結果を返す「対話型プログラム」の土台を作ります。
2. **入力とコマンドの扱い:** 入力された単なる文字列を、「コマンド」と「操作対象のデータ」に分割して解釈する仕組み（パーサ）を作ります。
3. **CRUD 操作とオンメモリ管理:** ハッシュテーブル（HashMap）を利用して、データを登録・取得・更新・削除（CRUD）するデータベースのコア機能を実装します。

このステップを踏むことで、まずは「動くデータベースの雛形」を手に入れることができます。さっそく、ユーザーとの対話の窓口となる REPL の作成から始めましょう。

2.2 対話型プログラム (REPL) の作成

データベースは、一度起動したらすぐに終了するプログラムではありません。ユーザーからの入力を待ち続け、入力された命令を実行し、その結果を表示することを繰り返します。例えば、多くのデータベースでは次のような対話形式で操作を行います。

```
> SELECT * FROM users;  
1|Alice|20  
2|Bob|18
```

このようなプログラムは **REPL (Read-Eval-Print Loop)** と呼ばれます。REPL とは、以下の 4 つの処理から構成されます。

- **Read** : ユーザーから入力を受け取る
- **Eval** : 入力された命令を実行する
- **Print** : 実行結果を表示する
- **Loop** : これらを繰り返す

本書で作成するデータベースも、この REPL 形式を採用します。

最初のデータベース

まずはデータベース本体となる `nekoDB` クラスを作成します。コンストラクタでは、データを保持するための `HashMap`、キーボード入力を受け取る `Scanner`、入力を解析するための `SimpleParser` を初期化します。

```
public nekoDB() {  
    db = new HashMap<>();  
    scanner = new Scanner(System.in);  
    parser = new SimpleParser();  
    System.out.println("Welcome to nekoDB!");  
}
```

現時点では、この `HashMap` がデータベースそのものになります。プログラムを起動すると、以下のメッセージが表示され、データベースの開始を知らせます。

```
Welcome to nekoDB!
```

REPL の実装

データベースが起動したら、ユーザーが終了を指示するまで入力を受け付け続けます。そのために、nekoDB クラスの start メソッドに無限ループを用意しています。

```
public void start() {
    while (true) {
        System.out.print("db > ");

        if (!scanner.hasNextLine()) {
            break;
        }

        String input = scanner.nextLine();
        // 後続の処理 (中略)
    }
}
```

このコードを実行すると、画面には次のように表示され、ユーザーの入力待ち状態になります。ここでユーザーがコマンドを入力すると、入力された文字列が1行分読み込まれます。

```
db >
```

2.3 入力とコマンドの扱い

前節ではユーザーからの入力を受け付ける REPL を作成しました。しかし、この時点では入力された文字列は単なる文字列であり、プログラムはその意味を理解できません。例えば、insert 1 Alice という入力があったとしても、プログラムにとっては "insert 1 Alice" という一つの文字列に過ぎません。本節では、この文字列を解釈するための処理を実装していきます。

データベースとして動作させるためには、REPL にて入力を受け付けた文字列を、

- 実行するコマンド
- 操作対象のデータ (引数)

に分けて解釈する必要があります。本書では、この役割を **SimpleParser** クラスが担当します。まず、入力された文字列を空白で分割する処理を SimpleParser クラスに実装

します。

```
public String[] parse(String sql) {
    String[] tokens = sql.trim().split("\\s+");
    tokens[0] = tokens[0].toLowerCase();
    return tokens;
}
```

入力の解析結果は次のようになり、コマンドと引数を個別に扱えるようになります。

入力	分割結果
insert 1 Alice	["insert", "1", "Alice"]
select	["select"]
delete 5	["delete", "5"]

データベースが最初知りたいのは、「どの操作を実行するのか」です。そこで、分割した配列の最初の要素をコマンド名として取り出します。

```
public String getCommand(String[] tokens) {
    return tokens.length > 0 ? tokens[0] : "";
}
```

例えば、["insert", "1", "Alice"] であれば、"insert"が返されます。これにより、以降の処理では入力全体を調べる必要がなくなり、コマンド名だけを見て処理を分岐できるようになります。

nekoDB クラスに戻り、取得したコマンド名をもとに処理を切り替えるよう実装します。ID は数値として扱うため、引数（文字列）を `Integer.parseInt()` で整数型に変換して各メソッドに渡します。

```
public void start() {
    while (true) {
        // ユーザーの入力待ち（中略）

        String[] tokens = parser.parse(scanner.nextLine());
        String command = parser.getCommand(tokens);

        // コマンドが空の場合はスキップ
        if (command.isEmpty()) {
            continue;
        }
    }
}
```



```
try {
    if (command.equals("insert") && tokens.length == 3) {
        int id = Integer.parseInt(tokens[1]);
        insert(id, tokens[2]);
    } else if (command.equals("select")) {
        if (tokens.length == 1) select();
        else if (tokens.length == 2) {
            int id = Integer.parseInt(tokens[1]);
            select(id);
        }
    } else if (command.equals("update") && tokens.length == 3) {
        int id = Integer.parseInt(tokens[1]);
        update(id, tokens[2]);
    } else if (command.equals("delete") && tokens.length == 2) {
        int id = Integer.parseInt(tokens[1]);
        delete(id);
    } else if (command.equals("exit")) {
        System.out.println("Bye!");
        break;
    } else {
        System.out.println("Unknown command");
    }
} catch (NumberFormatException e) {
    System.out.println("Error: ID must be a number.");
}
}
```

この分岐処理によって、例えば `insert 1 Alice` と入力すると、`tokens[1]` の "1" が数値の 1 に変換され、`insert(1, "Alice");` が呼び出されます（数値に変換できない文字が入力された場合はエラーを返します）。このように、入力された文字列を解析し、適切なメソッドへ処理を振り分けることで、データベースはさまざまな命令を実行できるようになります。

2.4 CRUD 操作の実装

前節までで、ユーザーが入力したコマンドを解析し、実行する処理を決定できるようになりました。しかし、この時点ではデータベースの実体としての機能はまだありません。そこで本節では、データベースの基本操作である **CRUD** を実装します。CRUD とは、データを扱うための 4 つの基本操作の頭文字を取ったものです。

操作	コマンド	説明
Create	insert	データを登録する
Read	select	データを取得する
Update	update	データを更新する

| Delete | delete | データを削除する |

多くのデータベースは、これら 4 つの操作を基本として構成されています。本書でも、まずはこの最小限の機能を実装していきます。

insert

新しいデータを登録するには、**insert** コマンドを使用します。

```
db > insert 1 Alice
```

insert メソッドでは、キー（ID）の重複チェックを行い、HashMap にキーと値の組み合わせを追加します。

```
public void insert(int id, String name) {
    if (db.putIfAbsent(id, name) != null) {
        System.out.println("Key already exists.");
    } else {
        System.out.println("Inserted.");
    }
}
```

select

登録したデータを確認するには、**select** コマンドを使用します。select コマンドには 2 つの使い方（引数なし、引数あり）があるため、メソッドをオーバーロード（同名で引数が違うメソッドを定義）して実装します。

1 つ目の使い方は、キーを指定せず、登録されているすべてのデータを取得します。

```
db > select
(1,Alice)
(2,Bob)
```

実装では、HashMap のすべての要素を順番に取り出して表示します。

```
public void select() {
    db.forEach((id, name) ->
```

第2章 最小のデータベース

```
        System.out.println("(" + id + ", " + name + ")");  
    }
```

2 つ目の使い方は、キー（ID）を指定し、そのデータだけを取得します。

```
db > select 2  
Bob
```

実装では、指定されたキーの存在を確認し、HashMap から取り出して表示します。

```
public void select(int id) {  
    if (!db.containsKey(id)) {  
        System.out.println("Record not found.");  
    } else {  
        System.out.println(db.get(id));  
    }  
}
```

update

登録済みのデータを変更するには、**update** コマンドを使用します。

```
db > update 2 Robert
```

update メソッドでは、まず対象のキーが存在するかを確認し、存在する場合のみ対応する値を上書きします。

```
public void update(int id, String name) {  
    if (!db.containsKey(id)) {  
        System.out.println("Record not found.");  
        return;  
    }  
    db.put(id, name);  
    System.out.println("Updated.");  
}
```

delete

不要になったデータは、**delete** コマンドで削除します。

```
db > delete 1
```

delete メソッドでも同様に、対象のキーが存在するか確認してから削除処理を行います。

```
public void delete(int id) {
    if (db.remove(id) == null) {
        System.out.println("Record not found.");
    } else {
        System.out.println("Deleted.");
    }
}
```

2.5 ハッシュテーブルによるオンメモリ管理

前節では、insert、select、update、delete の 4 つの基本操作を実装しました。これらの処理では、データを保存するための共通の仕組みとして Java の **Map** を利用しています。本節では、これがどのようなデータ構造であり、データベースの内部でどのような役割を果たしているのかを見ていきます。

ハッシュテーブル (Map) とは

Java の Map は、コンピュータサイエンスにおける「連想配列」や「ハッシュテーブル」と呼ばれる、キーと値のペアを管理するデータ構造の実装です。例えば、次のようなデータがあるとします。

キー(ID)	値(Name)
1	Alice
2	Bob
3	Carol

Map を使うと、この表のような対応関係をそのままプログラム上で表現できます。本書では、このキーを「レコードの ID」、値を「レコードの内容」として扱います。プログラムでは、データベース本体を次のように宣言しています。

```
public class nekoDB {
    // データベース本体 (キーは数値型のID、値は文字列)
    private Map<Integer, String> db;
```

```
public nekoDB() {
    db = new HashMap<>();
    // その他の初期化処理（中略）
}
// その他のメソッド（中略）
}
```

なぜ HashMap を使うのか

Map にはいくつかの実装種類がありますが、本書ではハッシュテーブルを利用した `HashMap` を採用しています。その最大の理由は、指定したキーに対応する値を「極めて高速に検索できる」ためです。

```
db > select 2
```

を実行すると、内部では `db.get(2);` が呼ばれます。`HashMap` は内部的にハッシュ関数という仕組みを使ってデータの保存場所を一瞬で特定するため、データ数が数件でも数十万件に増えても、キーを指定した検索にかかる時間はほとんど変わりません（計算量 $O(1)$ ）。

前節で実装した4つの操作は、すべてこの Map の基本メソッドに1対1で対応しています。

DB操作 Mapの操作
----- -----
insert putIfAbsent()
select get()
update put()
delete remove()

キーによる検索が圧倒的に速い `HashMap` は、このようにオンメモリ（メモリ上）で動作する小規模なデータベースの基盤として非常に適しているのです。

2.6 まとめと次章への課題

本章では、Java の `HashMap` を利用して、データベースの最小構成を実装しました。ユーザーからの入力を REPL で受け付け、コマンドを解析し、CRUD 操作を実行することで、「データを登録・取得・更新・削除する」というデータベースの基本機能を実現しました。ここまでの処理の流れをまとめると、次のようになります。



この構成はシンプルで理解しやすく、小規模なデータベースとして十分に機能します。しかし、データがメモリ上にしか存在しないため、プログラムを終了するとすべて失われてしまいます。実際のデータベースでは、サーバーを停止したりプログラムを終了したりしても、登録したデータは保持されなければなりません。

そのため、次章ではこの問題を解決するために、データをファイルへ保存する「永続化」の仕組みを実装します。これにより、プログラムを再起動してもデータを保持できる、本格的なデータベースへと一歩近づいていきます。

第3章

データの永続化

3.1 第2章までの課題：揮発的なデータ保持

第2章では、データを登録・取得・更新・削除できる最小限のデータベースを実装しました。このデータベースでは、ユーザーが入力したデータはすべて **HashMap** に保存されます。

しかし、この実装には大きな課題があります。それは、すべてのデータをメモリ上で管理しているため、プログラムを終了すると **HashMap** も同時に破棄され、データが消えてしまうという点です。

実際のデータベースでは、このようなことは起こりません。例えば、顧客管理システムでデータを登録したあと、パソコンの電源を切ったからといってデータが消えてしまったりは困ります。また、EC サイトで登録された商品情報や注文履歴も、システムを停止するたびに失われることはありません。このように、プログラムを終了したあともデータを保持し続ける性質を**永続化 (Persistence)** といいます。データベースにとって、永続化は最も重要な機能の一つです。本章では、このデータベースの永続化に取り組みます。

3.2 本章の方針：テキストファイルへのレコードの書き出し

「電源を切るとデータが消えてしまう」という課題を解決するために、本章ではメモリ上のデータをディスク（ファイル）に保存し、次回起動時に復元できる仕組みを実装します。

具体的には、以下の順序で実装を進めます。

1. **ファイルを扱う準備**: データベースのデータを保存するためのファイルと、その保存先ディレクトリを準備する仕組みを作ります。

2. **データの直列化（シリアライズ）設計**: メモリ上のデータ構造（HashMap）を、ファイルに書き込めるテキスト形式へと変換するルールを決めます。
3. **ファイルへの保存と復元の実装**: CRUD 操作でデータが変更されるたびにファイルへ書き出し、プログラム起動時にはファイルからデータを読み込んで復元する処理を組み込みます。

このステップを踏むことで、プログラムを再起動しても前回のデータが残っているという、データベースとしての必須条件を満たすことができます。まずは、データを保存するファイルの準備から見ていきましょう。

3.3 ファイルを扱う準備

前節では、データを永続化する必要性について説明しました。では、実際にはどこへデータを保存すればよいのでしょうか。本節では、最も単純な方法として、データをテキストファイルへ保存します。プログラムを終了してもファイルはディスク上に残るため、次回起動時に読み込むことでデータベースを復元できます。

まずは、データベースを保存するファイルを決めます。本書では、プロジェクト内の data ディレクトリにデータベースファイルを配置します。

```
project
├── src
│   └── data
│       └── chapter03.db
```

プログラムでは、保存先を定数として定義しています。

```
private static final String DATA_PATH = "data/chapter03.db";
```

Java ではファイルを扱う方法がいくつかありますが、本書では Java NIO が提供する Path クラスを利用します。以下のように、文字列として定義したパスから Path オブジェクトを生成し、ファイルの読み込みや書き込みもこれを利用して行います。

```
dataPath = Path.of(DATA_PATH);
```

ファイルへ保存するためには、保存先のディレクトリが存在している必要があります。

しかし、初めてプログラムを実行したときは、data ディレクトリが存在しない場合があります。そこで、まず保存先ディレクトリの有無を確認します。

```
if (!Files.exists(dataPath.getParent())) {  
    Files.createDirectories(dataPath.getParent());  
} else {  
    loadFromFile();  
}
```

これにより、保存先ディレクトリが存在しなければ自動的に作成され、ユーザーが事前に data ディレクトリを作成していなくても、プログラムが必要なディレクトリを準備できるようになります。一方で、保存先ディレクトリがすでに存在している場合は、以前に保存されたデータがある可能性があるため、loadFromFile() を呼び出し、保存されているデータを読み込みます。

ここまでで、データを保存するための準備が整いました。

3.4 データの直列化（シリアライズ）設計

前節では、データベースを保存するためのファイルを用意しました。しかし、ファイルを用意しただけではデータを保存できません。データを保存するためには、**どのような形式でデータを書き込むか**を決める必要があります。このように、メモリ上のデータ構造をファイルへ保存したり、ネットワークで送信したりできる形式（文字列やバイト列）に変換することを、**直列化（シリアライズ）**と呼びます。本節では、この直列化の形式を決めていきます。

現在のデータベースでは、レコードは `HashMap<Integer, String>` に保存されています。例えば、次のようなデータが登録されているとします。

ID	Name
--	-----
1	Alice
2	Bob
3	Carol

メモリ上では、

```
1 → Alice  
2 → Bob  
3 → Carol
```

という状態になっています。この内容をそのままの状態ではファイルへ保存することはできません。そこで、人間にも読みやすいテキスト形式へ変換（直列化）して保存します。

本節では、1 レコードをキー, 値という形式で保存します。例えば、1, Alice は、キー = 1, 値 = Alice という意味になります。複数のレコードがある場合は、以下のように 1 レコードを 1 行として保存します。

```
1, Alice  
2, Bob  
3, Carol
```

これが、本章で利用するデータベースファイルの内容です。CSV（Comma-Separated Values）のように項目をカンマで区切り、1 行を 1 レコードとして表しています。

3.5 ファイルへの保存

前節では、データベースファイルの保存形式として、キー, 値のテキスト形式を採用しました。しかし、保存形式を決めただけでは、データベースの内容はファイルへ反映されません。データベースへレコードを追加・更新・削除したときに、その内容をファイルへ書き出す処理が必要になります。また、データベースを起動したときに、ファイルに保存されたデータベースの内容を読み込む処理も必要です。本節では、その役割を担う `saveToFile()` と `loadFromFile()` メソッドを実装します。

保存と読み込みの実装

`saveToFile()` では、まず `HashMap` の全要素について、各要素をキー, 値形式の文字列へ変換します。そして、それらをリストにまとめ、ファイルへ書き出します。この実装では、ファイル内に各レコードの位置やサイズを管理する仕組みがないため、特定の 1 件のレコードだけを直接上書きすることはできません。そのため、1 件のレコードを更新または削除した場合でも、変更後の `HashMap` 全体を再度ファイルへ書き出し、ファイル全体を上書きする必要があります。

```
/** すべてのレコードを key,value 形式のテキストでファイルに保存する */
private void saveToFile() {
    try {
        List<String> lines =
            db.entrySet().stream()
                .map(entry -> entry.getKey() + "," + entry.getValue())
                .toList();
        Files.write(dataPath, lines);
    } catch (IOException e) {
        System.out.println("Failed to save database.");
    }
}
```

起動時（読み込み）には、逆の処理を行います。まず、ファイルを1行ずつ読み込み、各行をカンマで分割します。キーは文字列のままでは利用できないため、整数に変換してHashMapに追加します。不正なフォーマットの行が混ざっていた場合は例外をキャッチし、スキップすることでエラーによる強制終了を防ぎます。これにより、前回終了時と同じ状態のデータベースを安全に復元できます。

```
/** ファイルから key,value 形式のテキストを読み込んでレコードを復元する */
private void loadFromFile() {
    List<String> lines;
    try {
        lines = Files.readAllLines(dataPath);
    } catch (IOException e) {
        System.out.println("Failed to load database.");
        return;
    }

    for (String line : lines) {
        if (line.isBlank()) {
            continue;
        }
        try {
            String[] parts = line.split(",");
            db.put(Integer.parseInt(parts[0]), parts[1]);
        } catch (ArrayIndexOutOfBoundsException | NumberFormatException e) {
            System.out.println("Skipped invalid record format: " + line);
        }
    }

    System.out.println("Loaded records from file.");
}
```

CRUD 操作と保存処理

データベースの内容が変化するのは、

- insert

- update
- delete

の3つの操作です。そのため、それぞれのメソッドの最後で `saveToFile()` を呼び出します。例えば、`insert()` では、

```
db.putIfAbsent(id, value);
saveToFile();
```

という流れになっています。一方、`select` はデータを参照するだけで内容を変更しないため、保存処理は不要です。

本書では、データを変更するたびにファイルへ保存しています。このようにすることで、プログラムが途中で終了しても、直前までのデータが確実にディスクへ保存されています。

一方、実際のデータベースシステムでは、更新のたびにデータファイルへ直接書き込むとディスク I/O が頻発し、処理性能が低下する可能性があります。そのため、更新対象のデータを一度メモリ上のバッファに保持して変更し、変更されたデータページを適切なタイミングでディスクへまとめて書き戻す仕組みが一般的に用いられます。さらに、障害発生時にもデータの整合性を保つため、変更内容は通常、データページを書き戻す前にトランザクションログ (WAL) へ記録されます。これにより、システム障害後でもログを用いて未反映の更新を再実行したり、途中の更新を取り消したりすることができます。

本書では、永続化の基本的な考え方と処理の流れを理解しやすくするため、こうしたバッファ管理やログによる復旧機構は扱わず、更新のたびにファイル全体へ保存するシンプルな方式を採用しています。

3.6 永続化の確認

前節までで、データベースへ永続化の仕組みを追加しました。これにより、データは `HashMap` に保存されるだけでなく、ディスク上のファイルにも保存されるようになりました。本節では、実際にデータベースを操作し、プログラムを再起動したあともデータが保持されていることを確認します。

まずはデータベースを起動し、いくつかのレコードを登録します。

```
db > insert 1 Alice
db > insert 2 Bob
```

第3章 データの永続化

この時点で、メモリ上の HashMap だけでなく、データベースファイルにも同じ内容が保存されています。保存されたファイルをテキストエディタで開くと、次のような内容になっています。

```
1,Alice
2,Bob
```

1 行が 1 レコードとなっており、キー,値という形式で保存されていることが確認できます。プログラム内部では HashMap として管理されていますが、ファイル上ではシンプルなテキスト形式へ変換されていることが分かります。

次に、データベースを終了します。

```
db > exit
```

ここでプログラムは終了しますが、ファイルはディスク上に残っています。その後、再びデータベースを起動します。起動後に、

```
db > select
```

を実行すると、

```
(1,Alice)
(2,Bob)
```

と表示されます。これは、第 1 章のときとは異なり、前回終了時のデータが正しく復元されたことを意味しています。

3.7 まとめと次章への課題

本章では、第 2 章で作成したメモリ上のデータベースに、**永続化**の仕組みを追加しました。第 2 章では、データは HashMap のみに保存されていたため、プログラムを終了するとすべて失われていました。そこで本章では、この問題を解決するためにファイルを利用してデータを保存・復元する仕組みを実装しました。この構成によって、データベースを終了しても、登録したデータを保持できるようになりました。

一方で、第3章の実装には新たな問題があります。現在の `saveToFile()` では、データが1件だけ変更された場合でも、データベースファイル全体を書き直しています。例えば、100万件のレコードが保存されている状態で、

```
db > update 100 Robert
```

を実行したとき、変更したいデータは1件だけです。しかし、実際には100万件すべてを文字列へ変換し、ファイル全体を書き換えています。

1件だけではなくファイル全体を書き換えなければならないのには、理由があります。ファイルは基本的に先頭から順番に読み書きを行うものであり、ファイル内に各レコードの位置やサイズを管理する仕組みがありません。そのため、テキスト形式でデータを保存すると、途中の1件だけをピンポイントで書き換えることが極めて難しくなるのです。

例えば、

```
1,Alice
2,Bob
3,Carol
```

というファイルがあったとします。これは、ファイル上では次のように改行文字を含めた連続した文字列（バイトの並び）として保存されています。

```
1,Alice\n2,Bob\n3,Carol\n
```

ここで、2,Bob を 2,Robert へ変更したい場合、文字数（データ長）が増えてしまいます。そのまま上書きすると、後続の 3,Carol のデータ位置が後ろにズレてしまい、データが破損します。その結果、ズレた以降のすべてのデータを書き直す、つまり「ファイル全体を書き直す」必要が生じてしまうのです。

データ量が少ないうちは問題ありませんが、レコード数が増えるにつれて、更新処理にかかる時間も非現実的なほど長くなってしまいます。実際のデータベースでは、このような非効率な方法は採用されていません。変更があった部分だけを書き換えられるように、データを一定サイズの単位で管理しています。

第4章では、この問題を解決するために、データをページ（Page）という固定サイズの単位で管理する仕組みを導入します。ページ単位で管理することで、

- 必要な部分だけを読み込める

- 必要な部分だけを書き換えられる
- データ量が増えても効率よく更新できる

という、本格的なデータベースとしての利点が得られます。

第4章

ページとスロットによるデータ配置の改善

4.1 第3章までの課題：ファイル全体の書き直し

第3章では、プログラムを終了してもデータが消えないように、レコードをファイルへ保存する仕組みを作りました。

一方で、insert, update, delete によりレコードを1件でも変更したら「ファイル全体を書き直している」という点が非効率です。10000件のレコードがある状態で1件を更新すると、変更のなかった9999件も含めてすべてを書き出すことになります。書き込む量はレコードの総数に比例して増えていくため、データが増えるほど1回の更新が重くなっていきます。

これは保存の単位がファイル全体になっているためです。データベースが扱うデータは、ファイルの先頭から末尾まで1本のテキストとして並んでいるだけなので、途中の1件だけを差し替える手段がありません。

4.2 本章の方針：ページ単位での読み書き

この課題を解決するには、保存の単位をファイル全体よりも小さくし、変更したい部分だけを読み書きできるようにする必要があります。

そこで本章ではページを導入し、更新のたびにファイル全体を書き直すのではなく、変更のあった部分だけを読み書きする実装へと作り替えます。具体的には、次の順序で進めます。

1. ページ単位の導入: ファイルを一定サイズの固定長の単位に区切り、目的の場所へ

直接アクセスできるようにします。

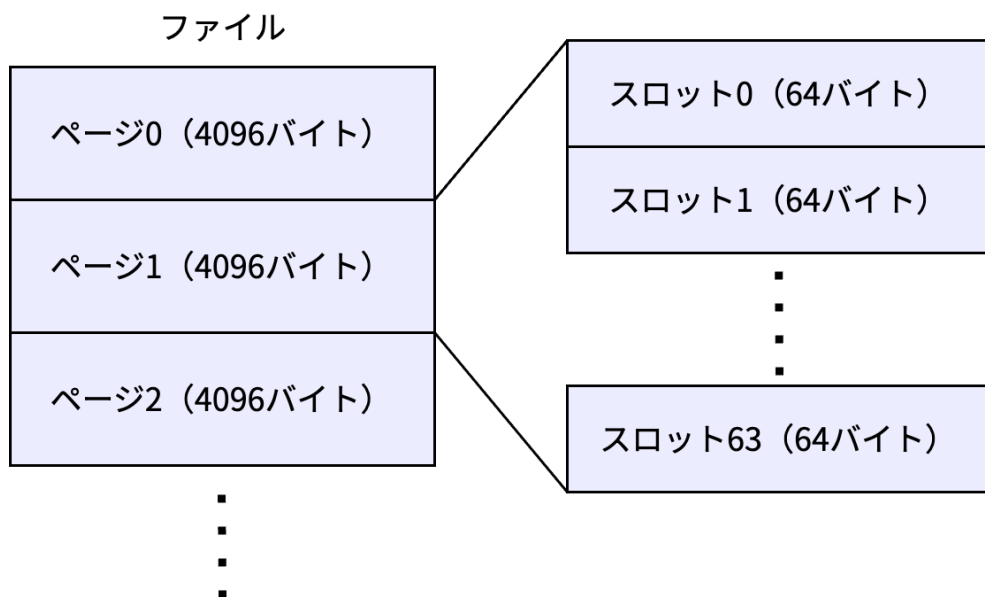
2. **スロットによるレコード設計**: 1 ページの中を「スロット」という固定長の区画に分割し、レコードを確実な位置へ格納できるようにします。
3. **CRUD の実装**: ページ単位の読み書きを用いて、必要な分だけのディスク I/O で CRUD 操作を完結させます。

この作り替えにより、レコードを 1 件変更するときにファイルへ書き込む量は、データの総数に関わらず「1 ページ分」で一定になります。次節ではまず、ページという単位の導入から見ていきます。

4.3 ページの導入と固定長スロットによるレコードの設計

ページとは、ファイルを一定の大きさに区切った固定長の単位のことです。本書では 1 ページを 4096 バイト (4KB) とし、ファイルの先頭から 4096 バイトずつ「ページ 0」「ページ 1」「ページ 2」……と区切って管理します。

固定長であることの最大のメリットは、計算によって位置を即座に特定できることです。あるページのファイルの先頭からのバイト位置 (オフセット) は「ページ番号 × 4096」で計算できます。そのため、Java の `RandomAccessFile` の `seek()` メソッドを使えば、目的のページへ直接移動し、そのページだけを読み書きできるようになります。



▲図 4.1 ページとスロットの構成

さらに、1 ページ (4096 バイト) の中を、64 バイトずつの区画に細かく分割します。この 1 区画を**スロット**と呼び、1 つのスロットに 1 件のレコードを格納します。1 ページは $4096/64 = 64$ 個のスロットを持つことになります。

ページが固定長であることと同様に、スロットも固定長であることで、s 番目のスロットの位置は「 $s \times 64$ 」で即座に計算できます。スロットの 64 バイトの内訳を以下に示します。

1スロット (64バイト)			
1バイト	4バイト	4バイト	55バイト
有効フラグ	キー	値のサイズ	実際の値
0 = 空き 1 = 使用	整数 (int)	整数 (int)	文字列のバイト配列 ※ 余った部分は0埋め

▲図 4.2 スロットの内訳

先頭の 1 バイトは**有効フラグ**です。そのスロットが使用中か空きかを表し、レコードを読み書きするときはまずこのフラグを確認します。続く 4 バイトにキー（整数）、次の 4 バイトに値のバイト数、残りの 55 バイトに値の本体を格納します。1 + 4 + 4 + 55 でちょうど 64 バイトになります。

Java でこのバイト列を組み立てるときは、スロットの開始位置（スロット番号×スロットサイズ）を先頭として、フラグ・キー・長さ・本体の順にデータを書き込んでいきます。

▼ スロットへ 1 件のレコードを書き込む

```
int offset = targetSlot * SLOT_SIZE;
ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
bb.put((byte) 1); // 有効フラグをセット
bb.putInt(id); // キーをセット

byte[] valBytes = value.getBytes(StandardCharsets.UTF_8);
int len = Math.min(valBytes.length, 55); // 最大55バイトに制限
bb.putInt(len);
bb.put(valBytes, 0, len);
```

値は `Math.min(valBytes.length, 55)` で最大 55 バイトに切り詰めています。読み出すときは、保存しておいた「値の長さ」の分だけバイトを取り出して文字列に戻します。

4.4 ページ単位での読み書きの実装

スロットの設計が決まったので、CRUD 操作をページ単位の読み書きで実装していきます。4つのコマンドはどれも、**ページを読み込む → スロットを調べる（または書き換える） → 必要ならページを書き戻す**という共通の流れで動きます。書き込みが必要など

きでも、ファイルへ書き戻すのは変更のあった 1 ページだけです。

以降、コマンドごとに処理の流れを順を追って見ていきます。

insert

1. 現在のファイルサイズから、ページ数を求める。
2. すべてのページ・スロットを調べ、同じキーがすでに存在しないか確認し、最初に見つかった空きスロットの位置を覚えておく。
3. 空きスロットが 1 つも無ければ、新しいページを 1 枚増やしてその先頭スロットを使う。
4. 書き込み先のページを読み込み、スロットにレコードを書き込む。
5. そのページだけをファイルへ書き戻す。

▼ 重複チェックと空きスロットの探索

```
int numPages = (int) Math.ceil((double) file.length() / PAGE_SIZE);

int targetPage = -1;
int targetSlot = -1;

for (int p = 0; p < numPages; p++) {
    byte[] buf = new byte[PAGE_SIZE];
    file.seek((long) p * PAGE_SIZE);
    file.read(buf);

    for (int s = 0; s < MAX_SLOTS; s++) {
        int offset = s * SLOT_SIZE;
        if (buf[offset] == 1) { // 使用中スロット
            ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
            bb.get(); // 有効フラグをスキップ
            if (bb.getInt() == id) {
                System.out.println("Key already exists. Use update command to modify.");
                return;
            }
        } else if (buf[offset] == 0 && targetPage == -1) { // 最初の空きスロットを記録
            targetPage = p;
            targetSlot = s;
        }
    }
}

// 空きがない場合は新しいページを用意する
if (targetPage == -1) {
    targetPage = numPages;
    targetSlot = 0;
}
```

書き込み先が決まったら、そのページを読み込み、スロットにレコードを書き込みます。1 件を書き込む処理は前述の通り、有効フラグ・キー・値の長さ・値の本体の順に By

第4章 ページとスロットによるデータ配置の改善

teBuffer で並べるだけです。最後に、変更したページだけをファイルへ書き戻します。

▼ 対象ページを読み込み、書き込み、書き戻す

```
// 対象のページを読み込む（新規ページなら読み込みは不要）
byte[] buf = new byte[PAGE_SIZE];
if (targetPage < numPages) {
    file.seek((long) targetPage * PAGE_SIZE);
    file.read(buf);
}

// スロットにレコードを書き込む
int offset = targetSlot * SLOT_SIZE;
ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
bb.put((byte) 1); // 有効フラグ
bb.putInt(id); // キー
byte[] valBytes = value.getBytes(StandardCharsets.UTF_8);
int len = Math.min(valBytes.length, 55);
bb.putInt(len); // 値の長さ
bb.put(valBytes, 0, len); // 値の本体

// 対象ページだけをファイルに上書き保存
file.seek((long) targetPage * PAGE_SIZE);
file.write(buf);
```

第3章では更新のたびに全レコードを書き出していました、ここで最後にファイルへ書き込んでいるのは4096バイト、つまり1ページ分だけです。データベース全体に何万件のレコードがあっても、1回のinsertで書き込む量は常に1ページで一定になります。ファイル全体の書き直しという無駄は、これで解消できました。

select

フラグが0のスロットは空きなので飛ばし、使用中のスロットだけを表示します。

▼ 全件を表示する select

```
for (int p = 0; p < numPages; p++) {
    byte[] buf = new byte[PAGE_SIZE];
    file.seek((long) p * PAGE_SIZE);
    file.read(buf);

    for (int s = 0; s < MAX_SLOTS; s++) {
        int offset = s * SLOT_SIZE;
        if (buf[offset] == 1) { // 使用中スロットのみ読み取る
            ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
            bb.get(); // フラグスキップ

            int key = bb.getInt();
            int valLen = bb.getInt();
            byte[] valBytes = new byte[valLen];
            bb.get(valBytes);
            String value = new String(valBytes, StandardCharsets.UTF_8);
```

```

        System.out.println("(" + key + "," + value + ")");
    }
}
}

```

キーを指定した検索も、ページとスロットを順に調べていく流れは同じです。

▼ キーを指定した検索

```

for (int p = 0; p < numPages; p++) {
    byte[] buf = new byte[PAGE_SIZE];
    file.seek((long) p * PAGE_SIZE);
    file.read(buf);

    for (int s = 0; s < MAX_SLOTS; s++) {
        int offset = s * SLOT_SIZE;
        if (buf[offset] == 1) {
            ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
            bb.get(); // フラグスキップ
            int recordKey = bb.getInt();

            if (recordKey == id) {
                int valLen = bb.getInt();
                byte[] valBytes = new byte[valLen];
                bb.get(valBytes);
                String value = new String(valBytes, StandardCharsets.UTF_8);
                System.out.println(value);
                return;
            }
        }
    }
}
System.out.println("Record not found.");

```

使用中スロットのキーが指定したキーと一致すれば、値を読み出して表示し、すぐに `return` で探索を打ち切ります。最後まで調べても見つからなければ `Record not found` . と表示します。ここで注目したいのは、**目的のレコードが後ろのページにあるほど、それまでのページをすべて読み込んでから見つかる**という点です。この性質が、本章の最後で述べる課題につながります。

update

`update` 操作は、対象を見つけたら同じスロットに新しい値を上書きし、そのページだけをファイルへ書き戻します。レコードの位置は変わらないため、他のスロットに影響を与えません。

▼ レコードの値を上書きする

```
if (recordKey == id) {
    // 対象を見つけたら同じスロットに上書き
    byte[] valBytes = value.getBytes(StandardCharsets.UTF_8);
    int len = Math.min(valBytes.length, 55);

    bb.putInt(len);
    bb.put(valBytes, 0, len);

    // 対象ページだけを保存
    file.seek((long) p * PAGE_SIZE);
    file.write(buf);

    System.out.println("Updated correctly");
    return;
}
```

delete

スロットの中身を実際に消したり、後ろのレコードを前に詰めたりはしません。その代わり、**スロット先頭の有効フラグを0にするだけで削除とみなします**。これを**論理削除**と呼びます。

▼ 有効フラグを0にする論理削除

```
if (recordKey == id) {
    // 対象を見つけたら、先頭フラグを0にして論理削除
    buf[offset] = 0;

    file.seek((long) p * PAGE_SIZE);
    file.write(buf);

    System.out.println("Deleted and saved to disk.");
    return;
}
```

4.5 処理時間の計測とパフォーマンス評価

insert, update, delete のたびにレコード全体をファイルに書き出していた前章と、ページとスロットを導入した本章の処理時間を比較してみましょう。

▼表 4.1 実行時間の比較（1 万件のデータが存在する状態）

コマンド	前章（全件書き直し）	本章（ページ管理）
<code>select 5000</code>	0.315 ms	9.468 ms
<code>insert 10001 user10001</code>	16.114 ms	4.622 ms
<code>update 5000 test</code>	8.025 ms	3.751 ms
<code>delete 5000</code>	9.833 ms	2.378 ms

前章と比較し、insert, update, delete の処理時間が短くなっていることが確認できます。ファイル全体ではなく、1 ページ（4KB）だけの読み書きで済むようになった効果が明確に表れています。

一方で、select の処理時間は大幅に増加しています。これは、前章までは「起動時に全てのデータをメモリに載せていた」のに対し、本章からは「毎回ディスクからページを読み込んで探すようになった」ためです。ディスクからの I/O アクセスが毎回発生するようになったことが、検索が遅くなった最大の理由です。

4.6 まとめと次章への課題

本章では、データベースのデータ配置を次のように改善しました。

- ファイルを 4096 バイトのページという固定長の単位に区切り、ページの位置を「ページ番号 × 4096」で即座に計算可能とした。
- 1 ページを 64 バイトの固定長スロットに分割し、s 番目のスロットの位置を「s × 64」で計算可能とした。
- 挿入・更新・削除で書き込むのは、変更のあった **1 ページ分だけ**になり、データ更新のたびにファイル全体を書き直す無駄が解消された。

一方で、検索（select）には致命的な課題が残っています。ファイルの先頭から順に目的のキーが見つかるまですべてのページをディスクから読み込んでスロットを調べており、これを**線形探索（Linear Search）**と呼びます。レコード件数が 1 万件、100 万件と増えるごとに、1 件を探すために読み込むディスク I/O 回数が激増してしまい、実用的な速度ではなくなってしまいます。

次章（第 5 章）では、この検索速度の問題を根本から解決するために、データベースの心臓部とも言える **B-Tree（B 木）** インデックスを導入します。ファイルの先頭から順に探すのではなく、「探しているキーがファイルのどこ（どのページとスロット）にあるのか」を一足飛びに知る仕組みを組み込み、再び圧倒的な検索スピードを取り戻します。

第 5 章

B-Tree による検索の高速化と領域管理

5.1 第 4 章までの課題：線形探索の限界

第 4 章では、データを「ページ」という固定長のブロック単位で管理するようにデータベースを拡張しました。これにより、ディスクとの細かな入出力 (I/O) が減り、効率的にデータを読み書きできるようになりました。

しかし、現在の実装には検索パフォーマンスにおいて大きな課題が残っています。それは、特定のレコードを探す (select) 際や、更新・削除対象を見つける際に、ファイルの先頭から最後まで、すべてのページとスロットを順番に調べているという点です。

このような検索方法を線形探索 (Linear Search) と呼びます。線形探索の処理時間はデータの件数 N に比例する $O(N)$ となります。データが数十件程度であれば処理時間はごくわずかですが、数万件、数百万件と増えていくと、たった 1 件のデータを検索・更新するだけでも毎回膨大なディスク読み込み (フルスキャン) が発生し、実用的な速度ではなくなってしまいます。「なぜ遅いのか」の理由は、この「毎回すべてのデータに目を通してから」という点にあります。

5.2 本章の方針：インデックスによる直接アクセス

この「毎回すべてのデータを調べる」という線形探索の課題を解決するため、本章では検索に特化したデータ構造 (インデックス) を導入し、目的のデータが「ディスク上のどこにあるか」を一瞬で特定できるようにします。

具体的には、以下の順序で実装を進めます。

1. **B-Tree (B 木) の導入と設計:** 検索キーと物理的な位置 (RecordId) を紐づけるデータ構造として、ページ管理と相性の良い「B-Tree」を採用し、ノードの構造を定義します。
2. **B-Tree アルゴリズムの実装:** 木のバランスを常に保つため、検索処理に加え、ノードの分割 (Split) や統合 (Merge) を伴う挿入・削除アルゴリズムを実装します。
3. **起動時のインデックス構築と空き領域管理:** データベース起動時にファイルから B-Tree を構築し、同時に空きスロットを `freeList` として管理して、挿入時の I/O コストも削減します。
4. **CRUD 操作の直接アクセス化:** これまでループで全件走査していた CRUD 処理を、B-Tree を用いたピンポイントな直接アクセスへと書き換え、どれくらい高速化したかを計測します。

このステップを踏むことで、データ量が 1 万件、100 万件と増えても、常に安定した速度で読み書きができる本格的なストレージエンジンが完成します。次節からはまず、「なぜ専用のデータ構造が必要になるのか」という基本的な考え方から見ていきましょう。

5.3 検索を高速化するデータ構造の導入

特定のデータへ直接アクセス (ランダムアクセス) するためには、データ本体のファイルとは別に、「探しているデータがファイルのどこにあるのか」をあらかじめ整理したデータ構造 (対応表) を導入する必要があります。

100 万件のデータから特定の ID を探すために、巨大なデータ本体を先頭から読み込むのは非常に非効率です。キーと場所のペアだけを抽出したコンパクトな対応表を用意しておけば、本体を探索するコストを劇的に下げることができます。現実世界で例えるなら、分厚い専門書を 1 ページ目から順番に読むのではなく、巻末の「索引」を見てページ番号を調べ、直接そのページを開くのと同一アプローチです。

プログラム上では、以下のような「**検索キー**」と「**データの物理的な場所**」のペアを管理する対応表を作成してこれを実現します。

```
[ 検索キー ]      | [ データの物理的な場所 ]
ユーザーID: 10   | ページ2、スロット4
ユーザーID: 15   | ページ8、スロット1
ユーザーID: 8765 | ページ120、スロット3
...
```

ユーザーが「ID が 8765 のデータを検索したい」と要求した場合、データベースは以下のように動きます。

1. 巨大なデータ本体を読む前に、まずはこのコンパクトな対応表を探索し、「ID: 8765」を見つけ出します。
2. そこに書かれている物理的な場所（ページ 120、スロット 3）を取得します。
3. データ本体のファイルから、**ページ 120 だけをピンポイントでディスクから読み込み**、該当スロットのデータを取得します。

次節からは、この仕組みを Java のプログラムとして具体的にどのように実装していくのかを見ていきます。

5.4 B-Tree の採用：RecordId の導入

プログラム上でこの仕組みを実現するために、まずは物理的な場所を `RecordId` クラスとして表現します。そして、検索キーと `RecordId` のペアを管理するデータ構造として、配列や二分探索木ではなく **B-Tree (B 木)** を採用します。(※実際の商用データベースでは範囲検索に最適化された派生型「B+Tree」が主流ですが、本章ではその基礎となる B-Tree を実装します。また、正式な「インデックス」という概念については第 6 章以降で扱います。)

データ構造として配列やリストを使用すると、途中でデータを追加・削除するたびに全体をシフトさせる必要があり非効率です。また「二分探索木」は、1 つのノードに 1 つのキーしか持たないため、データが増えると木が「縦に深く」成長してしまい、ディスクに保存した場合にページ読み込み (I/O) 回数が膨大になります。

対して B-Tree には、第 4 章で学んだ「**ページ (ブロック)**」との相性が良いという決定的な強みがあります。具体的には以下の理由が挙げられます。

- **1 つのノードに複数のキーを持てる (マルチウェイ)** : B-Tree は 1 つのノードに数十～数百個のキーを配列として格納します。これを 1 ページのサイズ (4KB 等) に合わせることで、1 回のディスク読み込みで大量のキーを一気にメモリへ読み込みめます。
- **階層が極めて浅い (横に広い)** : 1 ノードに 100 個のキー (101 本の枝) を持てば、高さ 3 階層で 100 万件のデータを表現できます。わずか最大 3 回のディスク I/O で目的のデータに到達できます。
- **常にバランスを保つ (平衡木)** : データの増減に合わせて自動で木を再構成するため、常に一定の速度 ($O(\log N)$) が保証されます。

実装としては、ファイル上の「どこ」を示す位置情報を保持するために、ページ番号とスロット番号を持つ `RecordId` クラスを新しく定義します。

▼ 位置情報を表す RecordId

```

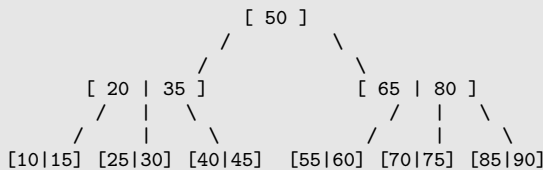
/** レコードの物理的な位置情報を保持する識別子 (Record ID) */
static class RecordId {
    int page;
    int slot;

    RecordId(int page, int slot) {
        this.page = page;
        this.slot = slot;
    }
}

```

また、B-Tree の全体構造とノードの中身を可視化すると以下の図のようになります。
(※見やすさのため、1 ノードあたりのキー数を 2 個までに絞っています)

▼ B-Tree の全体構造 (高さ 3 の例)



一番上にあるのが「ルートノード (根)」、中間が「内部ノード」、一番下が「リーフノード (葉)」です。各ノード内のキーが標識の役割を果たし、大小比較を行って進むべき枝を絞り込みながら末端へと到達するのが、B-Tree による検索メカニズムです。

5.5 B-Tree の実装

B-Tree をデータベースのデータ構造として機能させるには、検索だけでなく、更新操作に合わせて木構造のバランスを維持する仕組みが必要です。以降では、ノードの構造から順に、「検索」「挿入と分割」「削除と補充・統合」の各操作がどのような意図で実装されているのかを解説します。

ノードの表現 (BTreeNode) と最小次数

ノード (BTreeNode) の実装にあたっては、キーとポインタの配列を持たせます。さらに、「最小次数 (t)」と呼ばれるパラメータを導入し、1 ノードが持てる最大キー数を $2 * t - 1$ 、最小キー数を $t - 1$ と定義します。

なぜ最大キー数を $2 * t - 1$ とするという中途半端な数式にするのでしょうか。それは、ノードが満杯になって「2 つに分割 (Split)」される際の要件を美しく満たすため

す。満杯のノード ($2 * t - 1$ 個のキー) を左右に分割し、真ん中の 1 つのキーを親に渡すと、残された左右のノードのキー数はどちらもピッタリ「 $t - 1$ 」になります。分割直後でもノードが無駄なく最小基準を満たし、木全体のバランス (50% 以上の使用率) を維持できるように計算されているのです。

実際のデータを格納する「箱」であるノードは次のように定義されます。

▼ BTreeNode クラスの実装

```
/** B-tree ノード */
static class BTreeNode {
    int[] keys;           // キー配列 (ユーザーID)
    RecordId[] values;    // レコードポインタ配列 (保存場所)
    BTreeNode[] children; // 子ノードへのポインタ配列
    int numKeys;          // 現在のノードに入っているキーの数
    boolean isLeaf;       // 葉ノード (最下層) かどうか

    BTreeNode(int t, boolean isLeaf) {
        this.isLeaf = isLeaf;
        this.keys = new int[2 * t - 1];           // 最大キー数は  $2t - 1$ 
        this.values = new RecordId[2 * t - 1];    //
        this.children = new BTreeNode[2 * t];     // 最大子ポインタ数は  $2t$ 
        this.numKeys = 0;
    }
}
```

ノード内のキーが k 個あるとき、区切る枝の数は必ず $k + 1$ 本になるため、ポインタ配列 `children` の最大サイズはキーの最大数に 1 を足した $2 * t$ と定義されています。

検索処理 (Search)

目的のレコードを探す際は、ルートノードから開始し、各ノード内のキー配列を先頭から比較して探しているキーの範囲を見つけ出し、該当する子ノードへと再帰的に下っていくアプローチをとります。

ノード内での配列探索 (線形探索) はサイズが小さく CPU キャッシュ上で一瞬で終わるのに対し、ディスクから巨大なページを読むコストは膨大です。そのため、ディスク I/O の回数を「木の高さ (深さ)」の数回に抑えることが、データベースにおいて最も効率的な検索方法となります。

▼ BTree クラスの検索処理

```
public RecordId search(int key) {
    return root == null ? null : search(root, key);
}

private RecordId search(BTreeNode node, int key) {
    int i = 0;
```

```
// ノード内のキーを順番に比較し、進むべき枝（インデックス i）を探す
while (i < node.numKeys && key > node.keys[i]) i++;

// キーが一致すれば、その位置のRecordIdを返す
if (i < node.numKeys && key == node.keys[i]) {
    return node.values[i];
}

// 葉ノードまで到達して見つからなければ、存在しない
if (node.isLeaf) return null;

// 対象の子ノードへ進み、再帰的に検索を続ける
return search(node.children[i], key);
}
```

挿入とノードの分割 (Split)

データを挿入する際、対象のノードが最大キー数に達して満杯だった場合、そのままでは新しいデータを追加できません。そこで、ノードを 2 つに分割 (Split) し、真ん中のキーを親ノードに押し上げます。

これは、ノードの容量上限を超えないようにしつつ、「下から上へ」と木を成長させるためです。上へ持ち上がるように成長することで、すべての葉ノードまでの深さが常に均一に保たれ、バランスの崩れを防ぐことができます。

最小次数 $t=2$ (最大キー数 3) の満杯ノード [10 | 20 | 30] に分割が発生する様子を示します。

```
[ 分割前 ]
親ノード:      [ 50 ]
               /
満杯のノード: [ 10 | 20 | 30 ] <-- ここに分割処理(splitChild)を実行

[ 分割後 ]
真ん中の「20」が親へ押し上げられ、残りが左右の子になる
親ノード:      [ 20 | 50 ]
               /   \
新ノード左: [ 10 ]   新ノード右: [ 30 ]
```

実際のプログラムでは、「配列要素の右シフト (隙間づくり)」と「データの移動」によってこの分割処理を実現します。

▼ splitChild メソッドの実装 (一部省略)

```
void splitChild(BTreeNode parent, int i, BTreeNode fullNode) {
    // 1. 右半分を格納する新しいノードを作成
    BTreeNode newNode = new BTreeNode(t, fullNode.isLeaf);
    newNode.numKeys = t - 1;

    // 2. 満杯ノードの「右半分」のキーと値を、新ノードにコピー
    for (int j = 0; j < t - 1; j++) {
        newNode.keys[j] = fullNode.keys[j + t];
        newNode.values[j] = fullNode.values[j + t];
    }
    // ※内部ノードの場合は子ポインタ(children)のコピー処理も必要です

    fullNode.numKeys = t - 1; // 満杯ノードは「左半分」だけになる

    // 3. 親ノードのポインタ配列を右にズラし、新ノードを挿入する隙間を作る
    for (int j = parent.numKeys; j >= i + 1; j--) {
        parent.children[j + 1] = parent.children[j];
    }
    parent.children[i + 1] = newNode;

    // 4. 親ノードのキー配列も右にズラし、満杯ノードの「真ん中のキー」を押し上げる
    for (int j = parent.numKeys - 1; j >= i; j--) {
        parent.keys[j + 1] = parent.keys[j];
        parent.values[j + 1] = parent.values[j];
    }
    parent.keys[i] = fullNode.keys[t - 1]; // 真ん中のキーを親へ
    parent.values[i] = fullNode.values[t - 1];

    parent.numKeys++; // 親のキー数が1つ増える
}
```

削除処理におけるバランスの維持 (Borrow と Merge)

データを削除した結果、キーの数が最小基準 ($t - 1$) を下回った場合 (アンダーフロー)、隣接ノードからキーを移動させる「補充 (Borrow)」, または隣接ノードと 1 つにまとめる「統合 (Merge)」を行います。

これは B-Tree の厳格な最小キー数のルールを守るためです。この調整を行わないと、キーの少ないスカスカなノードが大量に発生してしまい、結果的に木が深くなって検索パフォーマンスが悪化してしまいます。

補充 (Borrow) では、大小関係を保つために直接横に移動させるのではなく、「親のキーを自分に降ろし、隣接ノードのキーを親に引き上げる」という回転動作を行います。

```
[ 補充前 ]
親ノード:           [ 40 ]
                  /   \
左の隣接ノード(余裕あり): 対象ノード(不足):
[ 10 | 20 | 30 ]         [ 50 ]
```

[補充後]
 親の「40」が対象ノードに降り、左隣接ノードの最大値「30」が親に上がる
 親ノード: [30]
 / \
 左の隣接ノード: [10 | 20] 対象ノード: [40 | 50]

▼ borrowFromPrev メソッドの実装

```
void borrowFromPrev(BTreeNode node, int idx) {
    BTreeNode child = node.children[idx];
    BTreeNode sibling = node.children[idx - 1]; // 左の隣接ノード

    // 1. childのキー配列を右に1つズラして空きを作る
    for (int i = child.numKeys - 1; i >= 0; --i) child.keys[i + 1] = child.keys[i];

    // 2. 親ノードのキーを、childの先頭に降ろす
    child.keys[0] = node.keys[idx - 1];

    // 3. 左の隣接ノード(sibling)の最後のキーを、親ノードに引き上げる
    node.keys[idx - 1] = sibling.keys[sibling.numKeys - 1];

    child.numKeys++;
    sibling.numKeys--;
}
```

一方、隣接ノードに余裕がない場合は、親の境界キーを降ろし、対象ノードと隣接ノードを1つにまとめる統合 (Merge) を行います。

▼ merge メソッドの実装

```
void merge(BTreeNode node, int idx) {
    BTreeNode child = node.children[idx];
    BTreeNode sibling = node.children[idx + 1]; // 右の隣接ノード

    // 1. 親のキーを、childの末尾に降ろす (これが両者をつなぐ境界線)
    child.keys[t - 1] = node.keys[idx];

    // 2. 右の隣接ノードのすべてのキーを、childにコピー
    for (int i = 0; i < sibling.numKeys; ++i) child.keys[i + t] = sibling.keys[i];
    child.numKeys += sibling.numKeys + 1;

    // 3. 親ノードの配列を左に詰めて隙間を塞ぐ
    for (int i = idx + 1; i < node.numKeys; ++i) node.keys[i - 1] = node.keys[i];
    for (int i = idx + 2; i <= node.numKeys; ++i) node.children[i - 1] = node.children[i];
    node.numKeys--;
}
```


5.6 起動時の B-Tree 構築と空き領域管理

現在のプログラムでは B-Tree がメモリ上にのみ存在するため、データベース終了時に揮発してしまいます。そのため、データベース起動時にファイル全体を 1 度だけ走査し、メモリ上に B-Tree を再構築する必要があります。また、その走査の過程でデータが存在しなかったスロットを `freeList` というキューに登録し、空き領域として管理します。

これまでの挿入 (`insert`) 処理では「空きスロットを探すための線形探索」がボトルネックになっていましたが、起動時に `freeList` を構築しておくことで、挿入時の I/O コストを削減し高速化を図る狙いがあります。

起動時に実行される `buildTree` メソッドで、B-Tree の再構築と `freeList` の構築を同時に行います。

▼ 起動時の B-Tree 構築と `freeList` 登録

```
private Queue<RecordId> freeList;

private void buildTree() throws IOException {
    int numPages = (int) Math.ceil((double) file.length() / PAGE_SIZE);
    int count = 0;

    for (int p = 0; p < numPages; p++) {
        byte[] buf = new byte[PAGE_SIZE];
        file.seek((long) p * PAGE_SIZE);
        file.read(buf);

        for (int s = 0; s < MAX_SLOTS; s++) {
            int offset = s * SLOT_SIZE;
            if (buf[offset] == 1) {
                // データが存在する場合はB-Treeに登録
                ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
                bb.get(); // フラグをスキップ
                int recordKey = bb.getInt();
                btree.insert(recordKey, new RecordId(p, s));
                count++;
            } else {
                // フラグが0（空きスロット）の場合はfreeListに追加
                freeList.offer(new RecordId(p, s));
            }
        }
    }
    System.out.println("B-Tree build complete. Loaded " + count + " records.");
}
```

5.7 B-Tree を用いた CRUD の高速化

構築した B-Tree と freeList を活用し、実際のデータベース操作 (select, insert, update, delete) の処理ロジックを、ループによる全件走査から直接アクセスへと書き換えます。

これにより、前章までの課題であった「線形探索によるパフォーマンス限界」を解消し、データ件数に依存しない $O(\log N)$ の安定した応答速度を実現します。

具体的には、検索・更新処理ではページ走査の for ループを削除し、btree.search(id) で取得した位置情報へ file.seek() で直接アクセスします。挿入処理では freeList.poll() から空き領域を即座に取得し、削除処理では空いた領域を freeList.offer() で再利用リストへ戻します。

▼ B-Tree を利用した select 処理の改善

```
public void select(int id) {
    try {
        // 1. B-Treeから位置情報 (RecordId) を取得
        RecordId rid = btree.search(id);
        if (rid == null) {
            System.out.println("Record not found.");
            return;
        }

        // 2. 対象のページのみをディスクから読み込む
        byte[] buf = new byte[PAGE_SIZE];
        file.seek((long) rid.page * PAGE_SIZE);
        file.read(buf);

        // 3. 対象スロットのデータを取得 (中略)
    } catch (IOException e) {
        System.out.println("Disk I/O Error during select.");
    }
}
```

▼ B-Tree と freeList を利用した insert 処理 (一部抜粋)

```
// B-treeによる高速な重複チェック  $O(\log N)$ 
if (btree.search(id) != null) {
    System.out.println("Key already exists.");
    return;
}

int targetPage = -1;
int targetSlot = -1;

// freeリストから空きスロットを即座に取得  $O(1)$ 
```

```

if (!freeList.isEmpty()) {
    RecordId freeId = freeList.poll();
    targetPage = freeId.page;
    targetSlot = freeId.slot;
} else {
    // freeリストが空の場合は新規ページを作成（中略）
}

```

実行速度の比較：どれくらい速くなったのか

ここまで実装してきた「B-Tree による検索の高速化」と「freeList による空き領域管理」によって、実際のデータベース操作がどれほど劇的に改善されたのかを確認してみましょう。

あらかじめデータベースに 1 万件のデータ (ID: 1~10000) を挿入した状態で、前章（線形探索）と本章（B-Tree + freeList）のプログラムに対して同じコマンドを実行し、処理時間を比較しました。

▼表 5.1 実行時間の比較（1 万件のデータが存在する状態）

コマンド	前章（線形探索）	本章（B-Tree + freeList）
select 8765	11.403 ms	0.686 ms
update 8765 test	5.933 ms	2.960 ms
delete 1	1.628 ms	0.991 ms
delete 8765	3.257 ms	0.983 ms
insert 10001 test	3.310 ms	1.218 ms

結果は一目瞭然です。ファイルの先頭から順にページを読み込んでいた select 処理は、B-Tree を導入したことで **16 倍以上もの高速化** を果たしています。

特に注目していただきたいのが delete の結果です。前章の線形探索では、ファイルの先頭付近にある delete 1 (1.628 ms) に比べて、末尾付近にある delete 8765 (3.257 ms) は、対象を見つけるまでに多くのデータを読み込む必要があり、処理時間が 2 倍近くかかっていました。しかし、本章の実装では delete 1 も delete 8765 も **どちらも約 0.9 ms** で完了しています。B-Tree は木がバランスを保つ（すべての葉までの深さが同じになる）ため、**データがファイルのどこにあっても、常に一定の速度 ($O(\log N)$) でアクセスできる**という強力な特性が数値として美しく表れています。

また、新しいデータを追加する insert についても、挿入前の「キーの重複チェック」が B-Tree で高速化され、さらに書き込み先の確保も「空きスロットを探す線形探索」から「freeList から位置情報を取り出すだけ ($O(1)$)」に変わったことで、処理時間が大幅に短縮されています。

5.8 まとめと次章への課題

本章では、データベースの検索・更新処理を高速化する「B-Tree」と、空き領域を効率的に再利用するための「freeList」を実装しました。

これらを導入したことにより、前章で直面した「線形探索によるパフォーマンスの限界」を見事に克服しました。検索対象のデータを特定する処理が $O(\log N)$ の時間計算量へと改善され、データ挿入時の空き領域探索にかかるディスク I/O も削減されました。これにより、内部的なデータアクセスの仕組みは実践的な速度で動作するようになりました。

しかし、データベースの「内部の処理速度」は向上したものの、ユーザーがデータベースを操作するための「インターフェース」には課題が残されています。

- **操作方法が複雑で扱いにくい:** 現在の仕様では、データの挿入や検索を行うために、専用のメソッドや独自の低レイヤーなコマンドを直接呼び出す必要があります。これではアプリケーションから汎用的かつ直感的に利用することができません。
- **「何を」取得するかは分かるが、「どう」実行するかが決まっていない:** 現在は「B-Tree の `search` を呼んでからファイルを読む」という単純な手順が決め打ちになっています。今後「特定の条件を満たすレコードだけを取得したい」といった要求が出た場合、ユーザー自身が「どの仕組みを使って、どの順番でデータを読み込むか」という具体的な手続き（手段）まで意識しなければならなくなってしまうます。

現代のリレーショナルデータベースが広く使われている最大の理由は、ユーザーが「欲しいデータ（目的）」だけを宣言的に記述すれば、データベース側が自動的に「最適な検索手順（手段）」を判断して実行してくれる点にあります。

次章（第 6 章）では、この課題を解決するために **SQL** の処理エンジンをデータベースに組み込みます。「何をするか」を記述する標準言語である SQL を読み解くため、字句解析（トークナイズ）や構文解析（パース）、そして抽象構文木（AST）の構築といった、言語処理系の領域へと足を踏み入れます。

まずは第一歩として、**SELECT** 文や **INSERT** 文をプログラムが理解できる形に変換する仕組みを作っていきます。

第 6 章

テーブル定義と SQL の実行

6.1 第 5 章までの課題：独自コマンドの限界とスキーマの欠如

第 5 章「B-Tree を用いた CRUD の高速化」では、B-Tree から RecordId を検索し、目的のレコードへ直接アクセスする仕組みを実装しました。これにより、ページを先頭から調べる線形探索の課題は改善されました。

一方、第 5 章までのデータベースには、利用者から見た二つの大きな課題が残っています。一つ目は、`id` と `name` という決められたカラムを前提としていることです。別のカラムやデータ型を扱うには、保存形式とプログラム自体を変更しなければなりません。

二つ目は、データを操作するために独自コマンドの書き方と実行手順を知る必要があることです。利用者が「どのデータが欲しいか」だけでなく、検索メソッドの呼び出し方で意識する構造では、用途が増えるたびに操作方法も増えてしまいます。

6.2 本章の方針：SQL と任意のテーブル定義

これらの課題を解決するため、第 1 章のロードマップで示した通り、本章ではいよいよ「テーブル定義と SQL の実行」を導入します。

一般的なリレーショナルデータベースでは、テーブルごとにカラムの構成を自由に定義できます。たとえば、利用者を保存する `users` テーブルと、注文を保存する `orders` テーブルでは、必要なカラムが異なります。

▼ 構成の異なる二つのテーブル

```
CREATE TABLE users (  
  id INTEGER,  
  name STRING(20),  
  age INTEGER  
);  
  
CREATE TABLE orders (  
  id INTEGER,  
  user_id INTEGER,  
  amount DOUBLE  
);
```

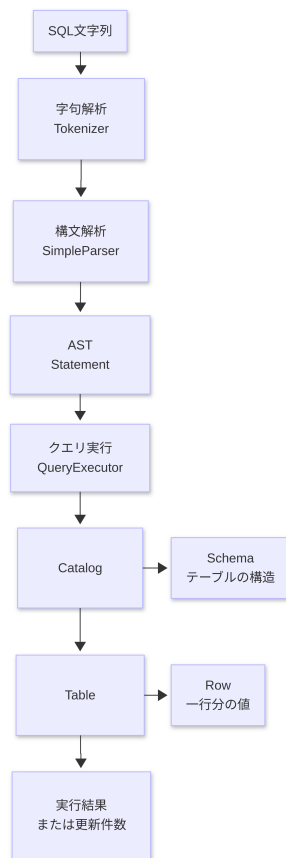
ユーザーがこのような標準的な SQL によって「欲しいデータ」を宣言的に記述し、データベース側がその内容を読み取って自動処理する入口を作るため、本章では以下の順序で実装を進めます。

1. **リレーショナルモデルと SQL 処理の導入**: まずは SQL を受け取ってから実行するまでの、処理全体のパイプラインを整理します。
2. **テーブル定義とカタログの設計**: ハードコードされたカラムを廃止し、任意のカラム構成 (Schema) を管理できる汎用的なテーブル構造を作ります。
3. **字句解析 (Tokenizer) の実装**: 入力された SQL 文字列を、プログラムが処理しやすい最小単位 (トークン) に分割します。
4. **構文解析 (Parser) と AST の構築**: トークンの並びから SQL の意味を読み取り、実行可能な「抽象構文木 (AST)」へと組み立てます。
5. **クエリエグゼキュータによる実行**: AST の内容に従って、汎用化されたテーブルに対して実際にデータの検索や更新を行います。

これらを実装することで、SQL 文字列の解析処理と、ストレージ (ページファイル) に対する読み書き処理が美しく分離され、私たちのプログラムは本格的な「リレーショナルデータベース」へと大きな進化を遂げます。

6.3 リレーショナルモデルと SQL 処理の導入

SQL の解析からデータの読み書きまでを一つの処理にまとめると、各クラスの役割や処理の境界が分かりにくくなります。そこで本節では、SQL を入力してから結果を表示するまでの流れを示し、後続の節で実装する各要素の役割を整理します。



▲図 6.1

`nekoDB.start()` は、REPL で一行ずつ SQL を受け取ります。REPL の入力ループとコマンド実行の基本構造は、第 2 章「対話型プログラム (REPL) の作成」と「入力とコマンドの扱い」で説明しています。本章では、SQL に対して字句解析と構文解析を行い、AST へ変換します。実装では `SimpleParser.parseStatement()` がこの解析処理の入口となり、生成した `Statement` を `QueryExecutor.execute()` へ渡します。

▼ SQL を解析して実行する処理

```
Statement statement = parser.parseStatement(sql);
executor.execute(statement);
```

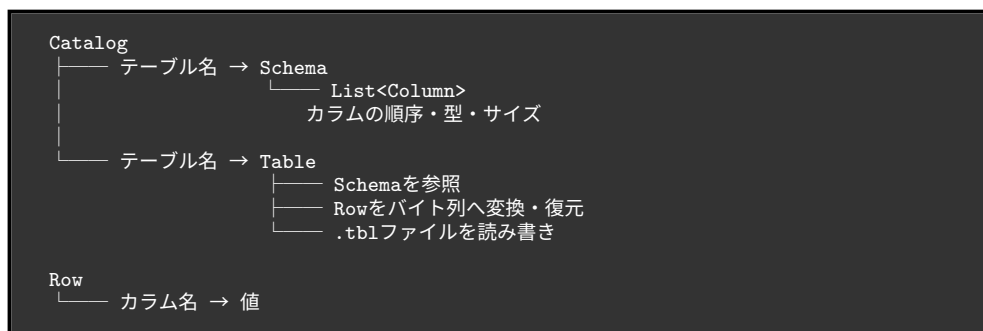
この時点で、SQL 文字列を直接扱う責務は字句解析と構文解析を行う Parser 側に限

定されます。クエリ実行以降の処理は、SQL の空白や括弧の位置を気にすることなく、AST である Statement が保持しているテーブル名、カラム名、値、条件を直接参照できるようになります。

本章で扱う SQL は、CREATE TABLE と CRUD に必要な INSERT、SELECT、UPDATE、DELETE です。SELECT では FROM、WHERE、一つの JOIN を扱います。SQL 規格のすべてを実装するのではなく、文字列が実行可能な構造へ変換される流れを確認できる範囲に限定して進めます。

6.4 テーブル定義とカタログの設計

任意のカラムを持つテーブルを扱うには、テーブルの定義と実際に行データを、固定の形式に依存せず表現する必要があります。この節では、必要な情報を Schema、Column、Row、Table、Catalog に分け、それぞれが一つの役割を担う方針でテーブルの構造を設計します。



Catalog はテーブル名から Schema と Table を取得できるようにします。Table は Schema を参照し、Row が持つ値をカラムの順序と型に従ってバイト列へ変換して .tbl ファイルへ保存します。読み込み時は逆に、ファイル内のバイト列を Schema に従って Row へ復元します。

Table：テーブルファイルの操作

Table は、Schema と RandomAccessFile を保持し、テーブルごとの .tbl ファイルを操作します。RandomAccessFile は、ファイル内の任意の位置へ移動して読み書きできる Java のクラスです。^{*1}4096 バイトのページ、固定長スロット、使用フラグによるレコード

^{*1} Oracle 「RandomAccessFile」: <https://docs.oracle.com/javase/jp/8/docs/api/java/io/RandomAccessFile.html>

管理については、第 4 章「ページという単位の導入」と「固定長スロットによるレコードの設計」で説明しています。本節では、それらの保存方式を繰り返さず、汎用的なテーブルを扱うために Table へ追加されたインターフェースを確認します。

Table が提供する主な操作は次のとおりです。

- `insert(Row)`：空きスロットへ行を保存します。
- `scan()`：有効なすべての行を返します。
- `scanRecords()`：行と `RecordId` の組を返します。
- `update(RecordId, Row)`：指定されたスロットを上書きします。
- `delete(RecordId)`：指定されたスロットの使用フラグを 0 にします。

`RecordId` はページ番号とスロット番号を保持します。この位置情報の目的と B-Tree との関係は、第 5 章「B-Tree の実装：RecordId の導入」で説明しています。本章の変更点は、`Row` と `RecordId` を一緒に返す `Record` を Table 内に定義し、`UPDATE` と `DELETE` から利用できるようにしたことです。

▼ Table が返す `RecordId` と `Record`

```
public record RecordId(int pageNo, int slotNo) {}  
  
public record Record(RecordId recordId, Row row) {}
```

`record` は、値を保持するためのクラスを簡潔に宣言できる Java の機能です。Java 16 から正式に導入されました。^{*2}

レコード先頭の使用フラグと、フラグを使った論理削除については、第 4 章「削除 (delete)」を参照してください。本章の Table も同じ方式を使用します。

Schema：コラム構成とレコード形式の管理

Schema は、テーブル名と Column の一覧を保持します。Table は Schema を参照することで、コラムの順序と各値のバイト数を判断できます。

▼ Schema の主要なフィールド

```
public class Schema {  
    private final String tableName;  
    private final List<Column> columns;  
    private static final int RECORD_HEADER_SIZE = 1;  
    // ...  
}
```

^{*2} Oracle 「レコード・クラス」：<https://docs.oracle.com/javase/jp/15/language/records.html>

固定長レコードからページ内のスロット数を求める考え方は、第4章「固定長スロットによるレコードの設計」で説明しています。第4章ではレコード形式が固定されていましたが、本章では Schema が Column の一覧からレコードサイズを動的に計算します。

▼ レコードサイズとスロット数の計算

```
public int getRecordSize() {
    int size = RECORD_HEADER_SIZE;

    for (Column column : columns) {
        size += column.size();
    }

    return size;
}

public int getMaxSlots(int pageSize) {
    return pageSize / getRecordSize();
}
```

Column：カラム名、データ型、サイズの定義

Column は Schema 内の record として定義され、カラム名、データ型、文字列の最大長を保持します。

▼ Column と DataType

```
public record Column(String name, DataType type, int length) {
    public int size() {
        return switch (type) {
            case INTEGER -> 4;
            case FLOAT -> 4;
            case DOUBLE -> 8;
            case STRING -> 4 + length;
        };
    }
}

public enum DataType {
    INTEGER,
    STRING,
    FLOAT,
    DOUBLE
}
```

STRING は、実際に保存した文字列の長さを示す 4 バイトと、定義時に指定した最大長の領域を使用します。たとえば STRING(20) のサイズは 24 バイト（長さを保持する 4 バイト + 最大 20 バイト）となります。

Row：一行分のデータの表現

Row は、カラム名と値の対応を `LinkedHashMap<String, Object>` で保持します。`LinkedHashMap` は、キーと値の対応を管理しながら、要素が追加された順序も維持する Map 実装です。^{*3} この性質により、Row へ値を追加した順序でカラムを取り出せます。

▼ Row への値の保存と取得

```
public void put(String columnName, Object value) {
    values.put(columnName, value);
}

public Object get(String columnName) {
    return values.get(columnName);
}
```

Row の値は Java 上では `Object` ですが、ファイルへ保存するときは `Schema.Column` の型に従って `INTEGER`、`FLOAT`、`DOUBLE`、`STRING` のバイト列へ変換されます。Java 上の値を保存形式へ変換し、読み込み時に復元する基本的な考え方は、第 3 章「データの直列化（シリアライズ）設計」で説明しています。本章では、固定されたキーと値ではなく、`Schema` のカラム順とデータ型を使って変換する点が異なります。

Catalog：テーブル定義と Table の管理

`Catalog` は、正規化したテーブル名をキーとして `Schema` と `Table` を管理します。

▼ Catalog が管理するマップ

```
private final Map<String, Schema> schemas = new HashMap<>();
private final Map<String, Table> tables = new HashMap<>();
```

`CREATE TABLE` を実行すると、`Catalog` は新しい `Table` を作成し、`Schema` と `Table` を各マップへ登録します。`Schema` の内容は `catalog.txt` へ保存され、行データはテーブル名に対応する `.tbl` ファイルへ保存されます。

```
data/
├── catalog.txt   テーブル定義（メタデータ）
├── users.tbl     users の行データ
└── orders.tbl    orders の行データ
```

^{*3} Oracle 「`LinkedHashMap`」: <https://docs.oracle.com/javase/jp/8/docs/api/java/util/LinkedHashMap.html>

catalog.txt の一行は、次のような形式です。

```
users|id:INTEGER:0,name:STRING:20,age:INTEGER:0
```

Catalog は起動時に catalog.txt を読み、Schema と Table を復元します。ファイルへの保存と起動時の復元による永続化については、第 3 章で説明しています。本章で新しく扱うのは、行データに加えて「テーブル定義」も永続化する点です。

6.5 字句解析 (Tokenizer) の実装

入力された SQL 文字列のままでは、実行処理がテーブル名や条件を項目ごとに参照できません。そこで、一般的なプログラミング言語のコンパイラと同じように、**字句解析**、**構文解析**、**AST の構築**という順序で SQL をプログラムが扱える形へ変換します。

まずは第一段階である「字句解析」から実装します。

字句解析とは

字句解析は、入力された文字列を、後続の構文解析が扱える最小単位へ分ける処理です。この最小単位を**トークン**と呼びます。本章では Tokenizer が字句解析を担当し、SQL 文字列をキーワード、識別子、リテラル、括弧、カンマ、比較演算子などのトークンへ分割します。

たとえば、構文解析が SQL を一文字ずつ読む場合、SELECT が一つのキーワードであり、>=が二文字で一つの演算子であることを毎回判断しなければなりません。字句解析を先に行うことで、構文解析は文字ではなく意味のある単位の並びに集中できます。

▼ 字句解析の対象となる SQL

```
SELECT users.name
FROM users
WHERE users.age >= 20;
```

この SQL は概念的に次のトークン列へ変換されます。終端のセミコロンはトークンへ追加されません。

```
[SELECT, users.name, FROM, users, WHERE, users.age, >=, 20]
```

Tokenizer はシングルクォートの内側を一つの文字列として扱います。そのため、'Taro Yamada' の途中に空白があっても分割されません。また、>=、<=、!= は二文字で一つの演算子になります。閉じていない文字列リテラルはエラーになります。

tokenize() は、入力文字列を先頭から一文字ずつ走査します。currentToken は現在組み立てているトークン、inString は現在位置が文字列リテラルの内側かどうかを表します。

▼ Tokenizer の走査処理

```
List<String> tokens = new ArrayList<>();
StringBuilder currentToken = new StringBuilder();
boolean inString = false;

for (int i = 0; i < sql.length(); i++) {
    char c = sql.charAt(i);

    if (c == '\\') {
        inString = !inString;
        currentToken.append(c);
        continue;
    }

    if (inString) {
        currentToken.append(c);
        continue;
    }

    // 文字列の外側にある文字を種類ごとに処理する
    // ...
}
```

シングルクォートを見つけるたびに inString を反転します。inString が true の間は、空白、カンマ、括弧も区切りとして扱わず currentToken へ追加します。たとえば 'Taro Yamada' はクォートを含む一つのトークンになります。

文字列の外側では、文字の種類に応じて次のように処理します。

▼ 区切り文字と演算子の処理

```
if (Character.isWhitespace(c)) {
    flush(currentToken, tokens);
} else if (c == '(' || c == ')' || c == ',') {
    flush(currentToken, tokens);
    tokens.add(String.valueOf(c));
} else if (c == ';') {
    flush(currentToken, tokens);
} else if (c == '=' || c == '<' || c == '>' || c == '!') {
    flush(currentToken, tokens);

    if (i + 1 < sql.length() && sql.charAt(i + 1) == '=') {
        tokens.add(String.valueOf(c) + "=");
    }
}
```

```

        i++;
    } else {
        tokens.add(String.valueOf(c));
    }
    } else {
        currentToken.append(c);
    }
}

```

空白または記号へ到達すると、そこまで currentToken へ蓄積した文字を flush() で tokens へ移します。括弧とカンマは構文解析で必要になるため、それ自体もトークンとして追加します。セミコロンは SQL の終端を示すだけなので追加しません。

比較演算子では、現在の文字の次が=かを確認します。次も演算子の一部なら二文字を結合し、走査位置 i を一つ進めます。この処理により、>と=ではなく>=という一つのトークンが生成されます。

▼ 組み立て中のトークンを確定する flush

```

private void flush(StringBuilder currentToken, List<String> tokens) {
    if (currentToken.length() > 0) {
        tokens.add(currentToken.toString());
        currentToken.setLength(0);
    }
}

```

入力の末尾まで走査した後も flush を呼び、最後のトークンを確定します。最後まで inString が true なら、対応する閉じクォートがないため例外を送出します。

たとえば、次の INSERT 文を走査するとします。

▼ Tokenizer の動作例

```
INSERT INTO users (id, name) VALUES (1, 'Taro Yamada');
```

主要な位置での状態は次のように変化します。

入力位置	currentToken	確定したtokens
INSERTの後の空白	"INSERT"	[INSERT]
usersの後の空白	"users"	[INSERT, INTO, users]
(	" "	[..., users, (]
'Taro Yamada'	"'Taro Yamada'"	空白を含めたまま保持
)	" "	[..., 'Taro Yamada',)]
;	" "	セミコロンは追加しない

最終的な戻り値は次のとおりです。

```
[INSERT, INTO, users, (, id, ,, name, ),  
VALUES, (, 1, ,, 'Taro Yamada', )]
```

6.6 構文解析 (Parser) と抽象構文木 (AST) の構築

字句解析で文字列をトークンのリストに分割できたら、次はその並び順を解析し、プログラムが実行可能な「構造」へと組み立てていきます。

構文解析 (SimpleParser)

構文解析は、字句解析で得たトークン列が SQL の文法に従って並んでいるかを確認し、その構造を読み取る処理です。たとえば SELECT 文なら、「SELECT の後に取得列があり、FROM の後にテーブル名がある」という並びを確認しながら、取得列、テーブル名、WHERE 条件などを取り出します。

本章では SimpleParser が構文解析を担当します。処理の入口である `parseStatement()` は、最初のトークンから SQL 文の種類を判定します。SELECT なら `parseSelect()`、INSERT なら `parseInsert()` というように、文法の異なる SQL を対応する解析メソッドへ振り分けます。

▼ SQL 文の種類を選択する処理

```
String command = tokens.get(0).toLowerCase();  
  
return switch (command) {  
    case "select" -> parseSelect(tokens);  
    case "insert" -> parseInsert(tokens);  
    case "create" -> parseCreateTable(tokens);  
    case "update" -> parseUpdate(tokens);  
    case "delete" -> parseDelete(tokens);  
    default -> throw new IllegalArgumentException(  
        "Unsupported SQL command: " + command);  
};
```

各解析メソッドは、トークンを参照する位置を `index` で管理します。値を一つ読み取るたびに `index` を進め、`expect()` を使って必要なキーワードや記号が所定の位置にあるか確認します。

▼ 必要なトークンを確認する expect

```
private void expect(List<String> tokens, int index, String expected) {  
    if (index >= tokens.size()) {  
        throw new IllegalArgumentException(  
            "Expected '" + expected + "', but reached end of SQL.");  
    }  
}
```

```

    }

    if (!tokens.get(index).equalsIgnoreCase(expected)) {
        throw new IllegalArgumentException(
            "Expected '" + expected
              + "', but found '" + tokens.get(index) + "'.");
    }
}

```

キーワードは大文字と小文字を区別せず照合します。想定したトークンがなければ解析を継続せず `IllegalArgumentException` を送出するため、不完全な構文が `QueryExecutor` へ渡ることはありません。

AST の表現 (Statement)

構文解析で読み取った内容は、後続の実行処理から参照できるデータとして保持する必要があります。その表現が**抽象構文木 (Abstract Syntax Tree, AST)**です。本章では `Statement` が AST に当たり、SQL 文の種類ごとにテーブル名、カラム名、値、条件などを保持します。

AST は、プログラムや SQL の構造を木として表したデータです。たとえば、次の二つの SQL は空白と大文字・小文字が異なります。

▼ 表記は異なるが意味が同じ SQL

```

SELECT name FROM users WHERE age >= 20;
select name from users where age>=20;

```

字句解析 (Tokenizer) が生成するトークンの表記には差が残りますが、構文解析 (SimpleParser) はどちらからも同じ構造の `Statement.Select` を生成します。クエリ実行を担う `QueryExecutor` は元の書き方を意識せず、`tableName` や `whereCondition` を参照できます。

一般的な AST では、文全体が根となり、その下に句や式のノードが接続されます。本章の実装では、`Statement` を根、`Condition` と `JoinClause` を子要素とする小さな木として表現できます。

```

Statement.Select
├── 選択列
├── FROMのテーブル
├── JoinClause
│   ├── 結合先テーブル
│   └── Condition (ON条件)
└── Condition (WHERE条件)

```


実装では専用の Node クラスを階層的に作る代わりに、Statement インターフェースを実装する Java の record を使います。record のフィールド間の包含関係が AST の親子関係に相当します。

▼ Statement の定義

```
public interface Statement {  
    record CreateTable(String tableName, List<Column> columns)  
        implements Statement {}  
  
    record Insert(String tableName,  
                  List<String> columnNames,  
                  List<String> values)  
        implements Statement {}  
  
    record Select(List<String> selectColumns,  
                  String tableName,  
                  Condition whereCondition,  
                  JoinClause joinClause)  
        implements Statement {}  
  
    record Update(String tableName,  
                  String columnName,  
                  String value,  
                  Condition whereCondition)  
        implements Statement {}  
  
    record Delete(String tableName, Condition whereCondition)  
        implements Statement {}  
}
```

たとえば、次の SQL を解析します。

▼ Statement.Select へ変換する SQL

```
SELECT name FROM users WHERE age >= 20;
```

生成される情報は次のとおりです。

```
Statement.Select  
├── selectColumns: [name]  
├── tableName: users  
├── whereCondition  
│   ├── left: age  
│   ├── operator: >=  
│   └── right: 20  
└── joinClause: null
```

Statement は元の SQL 文字列を保持するのではなく、後続の処理が参照する要素を項目ごとに保持します。この分離には次の利点があります。

- クエリ実行で SQL 文字列を再解析する必要がありません。
- 構文エラーを実行前に検出できます。
- SQL の表記が異なっても同じ実行処理を利用できます。
- プランナ (Planner) などの後続処理が条件やテーブル名を直接調べられます。

ここでは、CREATE TABLE に加えて、CRUD に対応する INSERT、SELECT、UPDATE、DELETE がどの Statement へ変換されるかを確認します。

CREATE TABLE の解析

CREATE TABLE の構文解析では、新しいテーブルを作るために必要なテーブル名とカラム定義を取り出します。CREATE TABLE は CRUD ではなく、テーブルを定義する DDL (Data Definition Language) です。

`parseCreateTable()` は CREATE、TABLE、テーブル名、開き括弧の順に読み取ります。その後、閉じ括弧へ到達するまでカラム名とデータ型を繰り返し取得します。

▼ カラム定義を読み取る処理

```
while (index < tokens.size() && !tokens.get(index).equals(" ")) {
    String columnName = tokens.get(index++);
    Schema.DataType type = parseDataType(tokens.get(index++));
    int length = 0;

    if (index < tokens.size() && tokens.get(index).equals("(")) {
        index++;
        length = Integer.parseInt(tokens.get(index++));
        expect(tokens, index, ",");
        index++;
    }

    columns.add(new Schema.Column(columnName, type, length));

    if (index < tokens.size() && tokens.get(index).equals(",")) {
        index++;
    }
}
```

データ型は `Schema.DataType` へ変換されます。STRING では `STRING(20)` のような括弧内の長さも取得し、`Statement.CreateTable` へ `Column` の一覧を保存します。

INSERT の解析

INSERT の構文解析では、挿入先のテーブル名、値を設定するカラム、実際に保存する値を取り出します。INSERT は CRUD の Create に当たる操作です。

`parseInsert()` は INSERT INTO の後からテーブル名を取得します。続くトークンが開き括弧ならカラム名一覧を読み、VALUES の括弧内から値一覧を読み取ります。

▼ INSERT からカラム名と値を取得する処理

```
String tableName = tokens.get(2);
List<String> columnNames = new ArrayList<>();

if (tokens.get(index).equals("(")) {
    index++;
    while (!tokens.get(index).equals(")")) {
        if (!tokens.get(index).equals(",")) {
            columnNames.add(tokens.get(index));
        }
        index++;
    }
    index++;
}

expect(tokens, index, "values");
// valuesの括弧内を同じ要領でvaluesへ追加する
```

カラム名を省略した場合、columnNames は空の一覧になります。値と Schema のカラムを対応付け、型変換する処理は QueryExecutor が担当します。

SELECT の解析

SELECT の構文解析では、結果に含めるカラム、読み取り元のテーブル、JOIN と WHERE の条件を取り出します。SELECT は CRUD の Read に当たる操作です。

parseSelect() は FROM へ到達するまでのトークンを取得列として集め、その直後のトークンをテーブル名として保存します。

▼ SELECT 列と FROM のテーブルを取得する処理

```
while (index < tokens.size())
    && !tokens.get(index).equalsIgnoreCase("from")) {
    if (!tokens.get(index).equals(",")) {
        selectColumns.add(tokens.get(index));
    }
    index++;
}

expect(tokens, index, "from");
index++;
String tableName = tokens.get(index++);
```

FROM 句の後に JOIN があれば JoinClause を、WHERE があれば Condition を読み取ります。本章の構文では、JOIN は一つだけで、JOIN の後に WHERE を書く順序に限られます。

UPDATE の解析

UPDATE の構文解析では、更新するテーブル、変更対象のカラム、新しい値、更新する行を絞り込む条件を取り出します。UPDATE は CRUD の Update に当たる操作です。

テーブル名に続いて SET を確認し、更新対象のカラム名、等号、新しい値を順に読み取ります。WHERE があれば Condition も取得します。

▼ UPDATE の主要要素を取得する処理

```
String tableName = tokens.get(1);
expect(tokens, 2, "set");

String columnName = tokens.get(3);
expect(tokens, 4, "=");
String value = tokens.get(5);

// 後続にWHEREがあればConditionを読み取る
```

構文解析の段階では、新しい値をまだ Column の型へ変換しません。QueryExecutor が Schema を参照して変換します。

DELETE の解析

DELETE の構文解析では、削除対象のテーブルと、削除する行を絞り込む条件を取り出します。DELETE は CRUD の Delete に当たる操作です。

parseDelete() は DELETE FROM に続くテーブル名を取得し、WHERE があれば Condition を読み取ります。

▼ DELETE の主要要素を取得する処理

```
expect(tokens, 0, "delete");
expect(tokens, 1, "from");
String tableName = tokens.get(2);

if (index < tokens.size()
    && tokens.get(index).equalsIgnoreCase("where")) {
    index++;
    whereCondition = parseCondition(tokens, index);
}
```

WHERE を省略した Statement.Delete では whereCondition が null になります。QueryExecutor では null の条件を常に一致すると判断するため、実行時には全行が削除対象になります。

構文解析は SQL の構造を読み取りますが、テーブルやカラムが実在するか、値を指定された型へ変換できるかまでは判断しません。これらは Catalog と Schema を参照でき

る QueryExecutor が確認します。

Condition と JoinClause

WHERE 句と JOIN の ON 句にも、比較対象と比較方法を表す構造が必要です。本章では、左辺、演算子、右辺の三要素を Condition で表し、JOIN に必要なテーブル名と ON 条件を JoinClause で表します。

▼ Condition と JoinClause

```
record Condition(String left, String operator, String right) {}  
record JoinClause(String tableName, Condition onCondition) {}
```

本章で扱う比較演算子は=、!=、>、>=、<、<=です。一つの Condition は一つの比較だけを表し、AND や OR による複合条件は扱いません。

JoinClause は、結合する右側のテーブル名と ON 条件を保持します。たとえば `users.id = orders.user_id` では、両辺がカラム名として解決されます。

Condition は `parseCondition()` で三つの連続したトークンから生成します。

▼ Condition を生成する処理

```
private Statement.Condition parseCondition(  
    List<String> tokens, int index) {  
    if (index + 2 >= tokens.size()) {  
        throw new IllegalArgumentException("Invalid condition.");  
    }  
  
    String left = tokens.get(index);  
    String operator = tokens.get(index + 1);  
    String right = tokens.get(index + 2);  
  
    return new Statement.Condition(left, operator, right);  
}
```

この実装では Condition の構文を三トークンに限定しています。そのため、括弧を使った条件や AND、OR、NOT は解析できません。この制限により、Parser と QueryExecutor の条件処理を小さく保っています。

6.7 クエリエグゼキュータによる AST の実行

クエリ実行は、構文解析が生成した AST を読み取り、実際のデータ操作へ変換する処理です。本章では QueryExecutor（クエリエグゼキュータ）がこの役割を担当します。

AST (Statement) だけでは、ページファイルに対する操作は行われません。本節では、Statement の種類ごとに実行処理を選択し、Catalog、Schema、Table を組み合わせて CRUD へ接続します。

QueryExecutor.execute() は Statement の実際の型を調べ、対応する実行メソッドを呼び出します。最小構成での処理手順は、第2章「CRUD 操作の実装」で説明しています。本章では CRUD 自体を再説明せず、SQL から作られた AST (Statement)、Schema、Catalog、Table を使って各操作を実行する部分に注目します。

▼ Statement に応じた処理の選択

```
if (statement instanceof Statement.CreateTable create) {
    executeCreateTable(create);
} else if (statement instanceof Statement.Insert insert) {
    executeInsert(insert);
} else if (statement instanceof Statement.Select select) {
    executeSelect(select);
} else if (statement instanceof Statement.Update update) {
    executeUpdate(update);
} else if (statement instanceof Statement.Delete delete) {
    executeDelete(delete);
}
```

この分岐により、Parser は SQL の字句解析と構文解析、QueryExecutor はクエリ実行、Table はファイル操作に集中できます。

CREATE TABLE と INSERT

CREATE TABLE では、Statement.CreateTable が保持するテーブル名と Column の一覧から Schema を作り、Catalog へ登録します。Catalog は空のテーブルファイルを作成し、catalog.txt を更新します。

INSERT では、Catalog から Schema と Table を取得します。buildRow() は Schema の各カラムを既定値で初期化した後、Statement.Insert の値を対応するカラムへ設定します。

値は Column の型に従って変換されます。

▼ 文字列からカラム型への変換

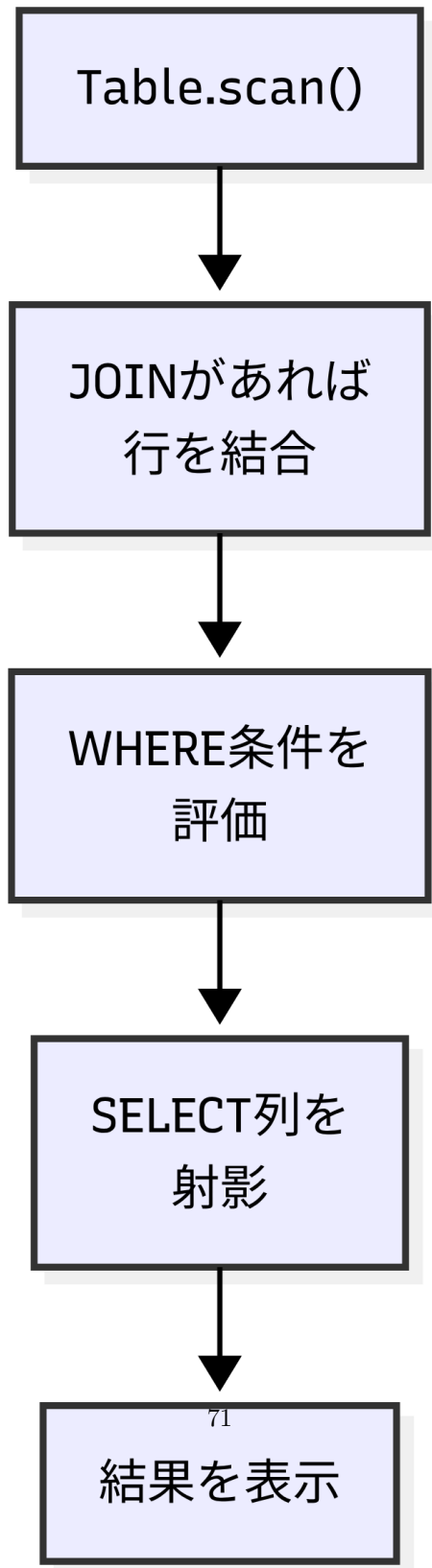
```
return switch (column.type()) {
    case INTEGER -> Integer.parseInt(value);
    case FLOAT -> Float.parseFloat(value);
    case DOUBLE -> Double.parseDouble(value);
    case STRING -> {
        if (value.length() > column.length()) {
            throw new IllegalArgumentException(
                "Value length exceeds column length.");
        }
    }
}
```

```
    }  
    yield value;  
  }  
};
```

Row が完成すると、Table は Schema のカラム順に値を直列化します。空きスロットの探索と、新しいページを追加する手順は、第 4 章で説明したものと同じです。本章では、その手順へ Schema に基づく可変のレコード形式を組み合わせています。

SELECT と JOIN

SELECT では、FROM 句の Table に対して `scan()` を呼び出します。WHERE 条件がある場合は、各 Row に対して `matches()` を実行し、一致した行だけを残します。その後、SELECT 句に指定されたカラムだけを `projectRow()` で取り出します。



JOIN がある場合は、左側の各行に対して右側の Table を走査する Nested Loop Join を実行します。カラム名の衝突を避けるため、結合中の Row では `users.id` のようにテーブル名を付けます。

条件の右辺は、最初にカラム名として解決されます。該当するカラムがなければ、整数、実数、文字列の順にリテラルとして解釈されます。数値同士は `double` へ変換して比較し、それ以外は文字列として比較します。

UPDATE と DELETE

UPDATE と DELETE は `scanRecords()` を使います。scanRecords は Row だけでなく RecordId も返すため、条件に一致した行の保存位置を特定できます。

UPDATE では対象カラムの存在を Schema で確認し、新しい値を Column の型へ変換します。条件に一致した Row を書き換え、同じ RecordId を指定して `Table.update` を呼び出します。

DELETE では条件に一致した RecordId を `Table.delete` へ渡します。同じスロットの上書きと、使用フラグによる論理削除の詳細は第 4 章を、削除したスロットの再利用は第 5 章を参照してください。本章では、これらの処理を SQL の WHERE 条件と接続しています。

WHERE 句を省略した場合、`matches(row, null)` は `true` を返します。そのため、UPDATE または DELETE の対象は全行になります。

6.8 実行結果のフォーマットと表示

ここまでに実装した SQL 処理が一連の流れとして動作するかを確認するには、各構文の入力と結果を対応付けて検証する必要があります。

本節では、`QueryExecutor` が処理結果を標準出力へ表示します。ここでは `users` と `orders` を作成し、INSERT、SELECT、FROM、WHERE、JOIN、UPDATE、DELETE の順に結果を確認します。

REPL は一行の入力を一つの SQL として解析するため、実行時には各 SQL を一行で入力します。また、実行時間は環境によって異なるため、以下の出力例では (`Executed in ... ms`) を省略します。

SQL が実行結果へ変わるまで

最初に、`SELECT name FROM users WHERE age >= 20;` が処理される過程をまとめます。各段階の内容は長さが異なるため、表ではなく箇条書きで示します。

- **SQL 入力文**：SELECT name FROM users WHERE age >= 20; を受け取ります。
- **字句解析 (Tokenizer)**：[SELECT, name, FROM, users, WHERE, age, >=, 20] というトークン列へ分割します。
- **構文解析 (SimpleParser)**：SELECT 文として、取得列 name、テーブル users、条件 age >= 20 を読み取ります。
- **AST (Statement)**：読み取った構造を Statement.Select として保持します。
- **クエリ実行 (QueryExecutor)**：AST に従ってテーブルを読み取り、{name=Taro} を表示します。

構文解析はトークン列から SQL の構造を読み取る処理であり、AST は読み取った構造を後続処理へ渡すためのデータ表現です。クエリ実行は AST を参照するため、元の SQL 文字列を再解析する必要がありません。

SQL コマンドと実行結果

実行例では、users に Taro と Hanako、orders にそれぞれの注文が登録されているものとします。

CREATE TABLE と INSERT では、作成したテーブル名と保存した Row が表示されます。

```
db > CREATE TABLE users (id INTEGER, name STRING(20), age INTEGER);
Table created: users

db > INSERT INTO users (id, name, age) VALUES (1, 'Taro', 20);
Inserted into users: {id=1, name=Taro, age=20}
```

SELECT では取得した Row を表示します。WHERE 条件を指定すると、一致する行だけが残ります。

```
db > SELECT * FROM users;
{id=1, name=Taro, age=20}
{id=2, name=Hanako, age=18}

db > SELECT name, age FROM users WHERE age >= 20;
{name=Taro, age=20}

db > SELECT name FROM users WHERE age > 100;
(empty)
```

JOIN では、ON 条件に一致する行を結合して表示します。

```
db > SELECT users.name FROM users JOIN orders ON users.id = orders.user_id;  
{users.name=Taro}  
{users.name=Hanako}
```

UPDATE と DELETE は、変更した行数を表示します。

```
db > UPDATE users SET age = 21 WHERE id = 1;  
Updated 1 row(s).  
  
db > DELETE FROM users WHERE id = 2;  
Deleted 1 row(s).
```

解析または実行中に例外が発生した場合は、Error: に続けてメッセージを表示します。

```
db > SELECT unknown FROM users;  
Error: Unknown column: unknown
```

WHERE 句を省略した UPDATE と DELETE はすべての行を対象にします。意図しない一括更新や一括削除を避けるため、実行例では WHERE 条件を指定しています。

REPL は成功時とエラー時のどちらでも、結果に続けて (Executed in ... ms) の形式で実行時間を表示します。

6.9 まとめと次章への課題

本章では、SQL 文字列を AST (Statement) へ変換し、テーブル定義に従ってクエリを実行する経路を構築しました。一方、現在のクエリ実行 (QueryExecutor) は SQL の条件に応じて実行方法を選択しません。

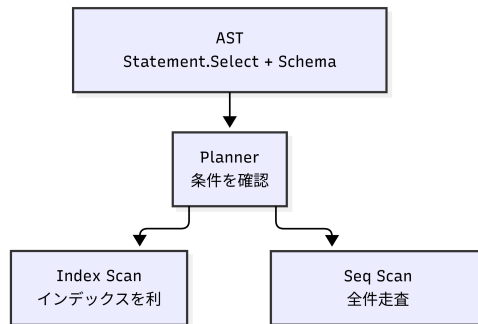
第 5 章「B-Tree を用いた CRUD の高速化」では、整数の id を B-Tree へ登録し、キーから RecordId を検索する方法を扱いました。本章では複数のテーブルと任意のカラムを扱うように構造を変更しており、QueryExecutor の SELECT は常に Table.scan() を呼び出します。したがって、本章のコードでは id を条件にした場合も、それ以外のカラムを条件にした場合も常に全件走査 (フルスキャン) になってしまいます。

インデックスを再び利用するには、少なくとも次の判断が必要です。

- WHERE 句があるか
- 条件の左辺に対応するカラムがあるか
- そのカラムにインデックスがあるか
- 比較演算子がインデックスで処理できるか

- インデックスを使わず全件走査する必要があるか

SQL の解析結果と Schema を参照し、最適な実行方法を決める役割がプランナ (Planner) です。たとえば、インデックスが設定された整数カラムに対する等価条件ならインデックス走査を選び、それ以外なら逐次走査を選ぶ、という判断を行います。



▲図 6.3

本章のクエリ実行には、この Planner がまだありません。次章（第 7 章）では、AST (Statement) から実行計画を作り、条件に応じてインデックス走査と全件走査を選択する仕組みを実装し、再び高速な検索を取り戻します。

第 7 章

プランナによる実行計画の選択

7.1 第 6 章までの課題：常に全件走査する SELECT

第 6 章で実装した SELECT 文は、FROM 句のテーブルに対して `Table.scan()` を呼び出し、全行をディスクから読み出してから WHERE 条件で絞り込んでいます。この方式では、条件指定の有無や内容に関わらず、常に全件走査（フルスキャン）となっています。

第 5 章では B-Tree による検索の高速化を実装しましたが、第 6 章の汎用的なテーブル構造にはまだ組み込まれていません。全件走査の処理時間はレコードの件数に比例するため、データ量が増加するとパフォーマンスが著しく悪化します。

インデックスによる検索を再び適切に行うためには、クエリを実行する前に少なくとも以下の判断を行う必要があります。

- WHERE 句が指定されているか
- 条件の左辺に対応するカラムが存在するか
- そのカラムにインデックスが設定されているか
- 比較演算子がインデックスで処理可能か

7.2 本章の方針：最適な実行経路の選択

この「常に全件走査してしまう」という課題を解決するため、本章では上記の条件を評価し、最も効率的な検索手順を自動的に決定する **プランナ (Planner)** を導入します。

具体的には、以下の順序で実装を進めます。

1. 実行計画とプランナの導入: 検索手順を決定する「プランナ」と、実際のデータ操

作を行う「クエリエグゼキュータ」の役割を明確に分離します。

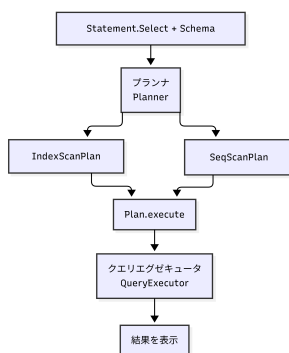
2. **カラムへのインデックス定義とカタログ保存:** CREATE TABLE でインデックスを指定できるようにし、テーブル定義（メタデータ）として保存・管理します。
3. **インデックスの構築と同期処理:** 第5章で作った B-Tree を利用し、データの追加・更新・削除に合わせてインデックスを自動で同期する仕組みを作ります。
4. **プランナによる実行方法（経路）の選択:** SQL の解析結果（Statement）とテーブル定義（Schema）から**実行計画（Execution Plan）**を生成し、全件走査（SeqScan）とインデックス検索（IndexScan）のどちらが最適かを判断するロジックを実装します。

このステップを踏むことで、ユーザーが内部のアルゴリズムを意識しなくても、データベース側が自動で B-Tree を活用して高速な検索を行ってくれるようになります。まずは、全体の処理の流れをどのように分離するかを見ていきましょう。

7.3 実行計画とプランナの導入

方針として、実行方法を決定する「プランナ」と、実際のデータ読み書きを担う「クエリエグゼキュータ」を明確に分離します。検索方法（アルゴリズム）が増加した際に、システム全体が複雑化するのを防ぐためです。

本章では、全件走査を表す SeqScanPlan と、インデックス検索を表す IndexScanPlan の2種類の実行計画（プラン）を用意します。システム全体の流れは以下のようになります。



▲図 7.1

第6章では QueryExecutor.executeSelect() が直接テーブルを走査していました

が、本章からはプランナにプラン（計画）を作成させ、エグゼキュータはそのプランをただ実行するだけの流れに変更します。

▼ プランを作って実行する executeSelect

```
private void executeSelect(Statement.Select statement) throws IOException {
    Schema schema = catalog.requireSchema(statement.tableName());
    Plan plan = planner.createPlan(statement, schema);
    plan.execute(this, statement);
}
```

7.4 カラムへのインデックス定義とカタログ保存

プランナがインデックス検索を選択できるように、テーブル定義（Schema）にインデックスの有無を含めます。

CREATE TABLE 文のカラム定義に INDEX キーワードを追加可能にし、Parser の構文解析処理を拡張します。

▼ INDEX キーワードを読み取る処理

```
int length = 0;
boolean indexed = false;

// STRING型の場合は括弧内の長さを読み取る（第6章と同じ）
if (index < tokens.size() && tokens.get(index).equals("(")) {
    // 長さの読み取り処理（省略）
}

// データ型の後ろにINDEXがあれば対象とする
if (index < tokens.size() && tokens.get(index).equalsIgnoreCase("index")) {
    indexed = true;
    index++;
}

columns.add(new Schema.Column(columnName, type, length, indexed));
```

これに合わせて、Schema.Column に indexed フラグを追加します。インデックスの有無は検索方法の決定に使用されるメタデータであるため、ディスク上のレコードサイズには影響しません。

テーブル定義を永続化する catalog.txt の保存フォーマットも変更します。末尾にインデックスの有無（1 または 0）を追加した 4 項目とします（※下記は読みやすくするため区切り文字の後にスペースを入れています）。

```
users | id:INTEGER:0:1, name:STRING:20:0, age:INTEGER:0:0
```

7.5 インデックスの構築と同期処理

インデックスは、キーから行を検索するための B-Tree をラップした `Index` クラスとして実装し、`Table` クラスごとに保持します。同一のキーを持つ複数行に対応するため、1つのキーに対して `List<Row>` を保持する構造とします。

▼ `Index` が公開する操作

```
public class Index {
    private final BTree btree;

    public void add(Object keyObj, Row row) { /* キーと行を登録する */ }
    public List<Row> search(Object keyObj) { /* キーに一致する行を返す */ }
    public void remove(Object keyObj, Row row) { /* 登録を取り消す */ }
}
```

インデックスのデータはディスクに保存せず、テーブルの起動時（ロード時）に実データを走査してメモリ上に再構築します。

▼ 起動時に `Index` を作り、既存行を登録する

```
private final Map<String, Index> indexes = new HashMap<>();

private void buildIndexes() throws IOException {
    for (Schema.Column column : schema.getColumns()) {
        if (column.isIndexed()) {
            indexes.put(column.name(), new Index());
        }
    }

    if (indexes.isEmpty()) return;

    for (Row row : scan()) {
        for (String col : indexes.keySet()) {
            Index idx = indexes.get(col);
            idx.add(row.get(col), row);
        }
    }
}
```

また、INSERT、UPDATE、DELETE によるデータ変更時には、ディスクへの書き込みと同時に対応する `Index` の更新処理も実行し、実データとの整合性を常に保ちます。

7.6 プランナによる実行方法（経路）の選択

プランナ（Planner）は、Statement とテーブル定義に基づき、SeqScanPlan か IndexScanPlan のどちらを実行するかを決定します。

▼ 条件を確認してプランを選ぶ createPlan

```
public Plan createPlan(Statement.Select statement, Schema schema) {
    Statement.Condition condition = statement.whereCondition();

    if (condition == null) {
        return new SeqScanPlan(statement.tableName(), null);
    }

    Schema.Column column = schema.getColumn(condition.left());

    if (column != null
        && column.isIndexed()
        && column.type() == Schema.DataType.INTEGER
        && condition.operator().equals("=")) {
        return new IndexScanPlan(statement.tableName(), condition);
    }

    return new SeqScanPlan(statement.tableName(), condition);
}
```

現在の実装では、IndexScanPlan が選択される条件は以下のすべてを満たす場合のみです。

- 対象カラムが INTEGER 型である
- 対象カラムに INDEX が指定されている
- 比較演算子が=である

本実装では範囲検索（>など）を含め、条件を満たさない場合はすべて全件走査（SeqScanPlan）を選択します。ただし、実際の商用データベースのように、インデックスのデータ構造として B-Tree の代わりに「B+Tree（葉ノード同士がポインタで連結された構造）」を採用していれば、葉ノードを辿るだけで済むため、範囲検索でもインデックスを活用して効率的にデータを走査することが可能になります。

7.7 計画フェーズと実行フェーズの分離

プラン（Plan）には決定された「実行方法の情報」のみを保持させ、実際のデータアクセス処理は ExecutionContext（本プログラムでは QueryExecutor が実装）に委譲します。

▼ Plan と ExecutionContext のインターフェース

```
public interface Plan {
    void execute(
        ExecutionContext context,
        Statement.Select statement
    ) throws IOException;
}

public interface ExecutionContext {
    void executeSeqScan(Statement.Select statement) throws IOException;
    void executeIndexScan(Statement.Select statement) throws IOException;
}
```

QueryExecutor 側の executeSeqScan() は、従来通りテーブル全行を読み込みます。一方の executeIndexScan() は、table.searchByIndex() を呼び出してインデックスから該当レコードのみをピンポイントで読み込みます。

▼ executeIndexScan (要点)

```
Statement.Condition condition = statement.whereCondition();
Schema.Column column = schema.getColumn(condition.left());

Object value = condition.right();
if (column != null) {
    value = parseValue(condition.right(), column);
}

List<Row> rows = table.searchByIndex(condition.left(), value);
// 取得した行に対して JOIN・WHERE を適用する処理 (省略)
```

7.8 実行例とプランの確認

id カラムにインデックスを設定したテーブルでの実行例を示します。

```
db > CREATE TABLE users (id INTEGER INDEX, name STRING(20), age INTEGER);
Table created: users

db > INSERT INTO users (id, name, age) VALUES (1, 'Taro', 20);
Inserted into users: {id=1, name=Taro, age=20}
db > INSERT INTO users (id, name, age) VALUES (2, 'Hanako', 18);
Inserted into users: {id=2, name=Hanako, age=18}
```

id を完全一致で検索すると、インデックス検索が選択されます。

```
db > SELECT * FROM users WHERE id = 2;
IndexScan : users
{id=2, name=Hanako, age=18}
```

インデックスのないカラム (age) や、範囲検索 (>) を指定した場合は全件走査が選択されます。

```
db > SELECT * FROM users WHERE age = 18;
SeqScan : users
{id=2, name=Hanako, age=18}

db > SELECT * FROM users WHERE id > 1;
SeqScan : users
{id=2, name=Hanako, age=18}
```

7.9 まとめと次章への課題

本章では以下の機能を実装しました。

- INDEX 定義のカタログへの永続化
- テーブルごとの B-Tree インデックス管理とデータ同期
- プランナ (Planner) による Statement と Schema に基づく実行計画の決定
- プラン (Plan) と実行処理 (ExecutionContext) の分離

これにより、SQL の条件に応じて賢くインデックス検索と全件走査を切り替えられるようになり、検索速度が大きく改善されました。

しかし、現在のクエリ実行の仕組みには、まだアーキテクチャ上の大きな欠陥があります。それは、現在の実装では、抽出したレコードをすべてメモリ上の List<Row> に一気に保持してから、絞り込みや結合を行っているという点です。データ量が増加すると、このリストがメモリを過大に消費し、いずれメモリ枯渇 (Out Of Memory) を引き起こしてデータベースがクラッシュしてしまいます。

次章 (第8章) では、このメモリ枯渇問題を解決するため、処理単位をオペレータ (Operator) として細かく分離し、1 行ずつデータをバケツリレーのように処理する「イテレータ (Volcano) モデル」を導入します。これにより、どれだけデータが増えてもメモリ消費を一定に抑えられる、堅牢なクエリ実行アーキテクチャへとシステムを改修します。

第 8 章

オペレータによる実行処理の分離

8.1 第 7 章までの課題：実行処理とストレージの密結合、メモリ枯渇

第 7 章では、SQL の条件に応じて全件走査とインデックス走査を選択するプランナ (Planner) を実装しました。しかし、WHERE による絞り込み、SELECT 列の取り出し、JOIN などの処理は依然としてクエリエグゼキュータ (QueryExecutor) 内に残っており、機能を増やすたびに分岐が肥大化してしまう密結合な構造でした。

さらに深刻なのが、パフォーマンスとメモリの課題です。現在の実装では、取得した行を `List<Row>` に一気に保持してから次の処理（絞り込みなど）へ渡すため、処理の途中に巨大な中間結果が作られます。データが増えるとメモリを過大に消費し、いずれメモリ枯渇 (Out Of Memory) を引き起こす危険性があります。

8.2 本章の方針：実行処理の抽象化とパイプライン化

これらの課題を解決するため、第 1 章のロードマップで示した通り、本章では「構造の分離」を行います。実行処理をオペレータ (Operator) という小さな単位へ分け、それらをパイプラインのようにつなぎ合わせるアーキテクチャを導入します。

具体的には、以下の順序で実装を進めます。

1. **イテレータモデルの導入**: すべての処理を `open()`、`next()`、`close()` という共通のインターフェースで扱えるように基盤を整えます。
2. **走査と加工のオペレータ化**: データの読み出し（走査）、WHERE による絞り込み (Filter)、SELECT 列の抽出 (Project)、結合 (JOIN) をそれぞれ独立したオペレータとして実装します。

3. **更新系のオペレータ化**: INSERT、UPDATE、DELETE も同じ仕組みで動くようにオペレータ化し、実行方法を完全に統一します。
4. **プランナによる実行計画ツリーの構築**: プランナがオペレータを組み合わせで「実行計画ツリー」を作り、クエリエグゼキュータが内部構造を意識せずに実行する仕組みへと改修します。

本章の目的は、単にクラスを増やすことではありません。行を 1 行ずつバケツリレーのように渡すことで巨大な中間結果をなくし、今後 SQL の機能を追加しても既存処理への影響を最小限に抑えられる、拡張性と堅牢性を備えた実行基盤を作ることです。

8.3 オペレータとイテレータ (Volcano) モデルの導入

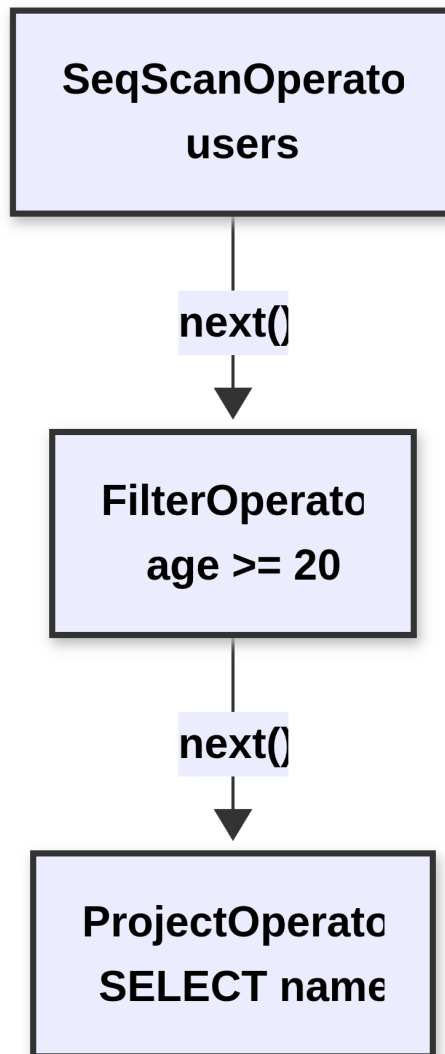
検索、絞り込み、射影、結合を一つのメソッドに書くと、処理順序を変更するたびにメソッド全体を修正しなければなりません。そこで、それぞれの役割をオペレータとして独立させ、上位のオペレータが下位のオペレータへ「次の 1 行をちょうだい」と要求するアーキテクチャを導入します。これはデータベースエンジンの世界で**イテレータモデル**（または **Volcano モデル**）と呼ばれる標準的な手法です。

たとえば、次の SELECT 文を考えます。

▼ 絞り込みと射影を行う SELECT 文

```
SELECT name FROM users WHERE age >= 20;
```

この SQL は、全件走査、条件による絞り込み、カラムの取り出しというオペレータをつないだ「実行計画ツリー」で表せます。



▲図 8.1

クエリエグゼキュータが根（トップ）の ProjectOperator へ行を要求すると、要求は FilterOperator、SeqScanOperator の順に下へ伝わります。一番下の SeqScanOperator が返した行を FilterOperator が判定し、条件を満たした行だけを ProjectOperator が {name=Taro} のような結果へ変換して上に返します。

すべてのオペレータを同じ方法で扱うため、次のインターフェースを定義します。

▼ Operator インターフェース

```
public interface Operator {  
    void open() throws IOException;  
  
    Row next() throws IOException;  
  
    void close() throws IOException;  
}
```

`open()` は処理の準備、`next()` は次の 1 行の取得、`close()` は終了処理を担当します。返せる行がなくなると `next()` は `null` を返します。

この約束を守れば、全件走査でもインデックス走査でも、呼び出し側から見た実行方法は変わりません。本章では、行を読み出す `SeqScanOperator` と `IndexScanOperator`、行を加工する `FilterOperator`、`ProjectOperator`、`NestedLoopJoinOperator`、データを変更する `InsertOperator`、`UpdateOperator`、`DeleteOperator` を順に実装していきます。

8.4 走査オペレータの実装

全件走査とインデックス走査を共通のオペレータとして扱えると、上位の絞り込みや射影処理を変更することなく、プランナが検索方法だけを差し替えることができます。本節では、実行計画ツリーの「葉（一番下）」にあたり、行を供給する二つのオペレータを作ります。

SeqScanOperator

`SeqScanOperator` は、指定されたテーブルの行を先頭から順に返します。`open()` でイテレータを準備し、`next()` が呼ばれるたびに次の `Row` を返します。

▼ SeqScanOperator の主要な処理

```
public void open() throws IOException {  
    List<Row> rows = table.scan();  
    this.iterator = rows.iterator();  
}  
  
public Row next() throws IOException {  
    if (iterator != null && iterator.hasNext()) {  
        Row rawRow = iterator.next();  
        return qualifyRow(tableName, schema, rawRow);  
    }  
    return null;  
}
```

```
public void close() throws IOException {
    this.iterator = null;
}
```

`qualifyRow()` は、カラム名 (`id`) をテーブル名付き (`users.id`) に変換して `Row` へ登録する処理です。これにより、単一テーブルでは短いカラム名を使いつつ、JOIN 時には同名カラムをテーブル名で区別できるようになります。

IndexScanOperator

`IndexScanOperator` は、第 7 章で実装した `Table.searchByIndex()` を利用して、インデックスに一致する行だけを取得します。`SeqScanOperator` と異なるのは、主に `open()` で取得対象を絞り込む点です。

▼ `IndexScanOperator` の `open`

```
public void open() throws IOException {
    Schema.Column column = schema.getColumn(condition.left());
    Object value = condition.right();

    String columnName =
        column != null ? column.name() : condition.left();
    if (column != null) {
        value = parseValue(condition.right(), column);
    }

    List<Row> rows = table.searchByIndex(columnName, value);
    this.iterator = rows.iterator();
}
```

`next()` と `close()` は `SeqScanOperator` と同じ形です。そのため、上位のオペレータはどちらの走査方法が使われているかを一切意識しません。

8.5 行を加工するオペレータの実装

WHERE、SELECT、JOIN といった処理を独立したオペレータへ分けることで、条件評価やカラム選択を複数の実行計画で再利用できるようになります。

FilterOperator

`FilterOperator` は、子オペレータ（下位）から受け取った行を評価し、WHERE 条件を満たす行だけを上に返します。

▼ 条件に一致するまで行を取得する処理


```
public Row next() throws IOException {
    while (true) {
        Row row = child.next();
        if (row == null) {
            return null; // 子からのデータが尽きた
        }
        if (ConditionEvaluator.matches(row, condition)) {
            return row; // 条件に一致した行だけを返す
        }
    }
}
```

条件評価は `ConditionEvaluator` という別クラスへ分離します。第 6 章ではクエリエグゼキュータ内にあった比較処理を独立させることで、`FilterOperator` だけでなく後述の更新系オペレータ等からも同じ判定ロジックを利用できます。

ProjectOperator

`ProjectOperator` は、子オペレータが返した `Row` を、`SELECT` 句で指定された形（特定のカラムのみ）へと変換（射影）します。

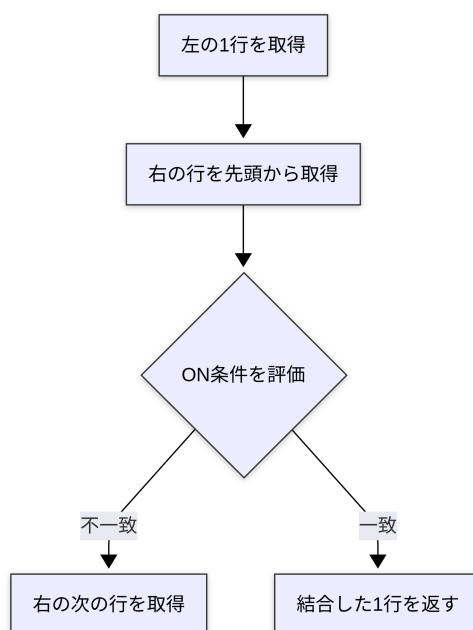
▼ 1 行を射影する処理

```
public Row next() throws IOException {
    Row row = child.next();
    if (row == null) {
        return null;
    }
    return projectRow(row, selectColumns, hasJoin);
}
```

`SELECT` 句が `*` なら全カラムを返し、カラム名が指定されていれば該当する値だけを抽出して返します。これを `FilterOperator` の上に配置することで、「`WHERE` で絞り込んでから `SELECT` 列を取り出す」という論理的な順序が実行計画ツリー上で表現されます。

NestedLoopJoinOperator

`NestedLoopJoinOperator` は左右の二つの子オペレータから行を受け取り、`ON` 条件を満たす組み合わせを返します。



▲図 8.2

右側を最後まで調べたら、右のオペレータを `close()` して `open()` し直し、左の次の行との比較を始めます。専用のオペレータにすることで、将来的に「Hash Join」などのより高速な結合アルゴリズムを追加する場合も、同じインターフェースで簡単に差し替えられます。

8.6 更新系オペレータの実装

`SELECT` だけをオペレータ化しても、`INSERT`、`UPDATE`、`DELETE` の個別処理がクエリエグゼキュータに残っているのは、アーキテクチャが統一されません。更新処理もオペレータにすることで、すべての CRUD を `open`、`next`、`close` で実行し、処理件数の数え方なども共通化できます。

InsertOperator

`InsertOperator` は、`Statement` の値を `Schema` に従って `Row` へ変換し、テーブルへ挿入します。`INSERT` は 1 回だけ実行すればよいので、`executed` フラグで二重実行を防ぎます。

▼ 一度だけ行を挿入する next

```
public Row next() throws IOException {
    if (executed) {
        return null;
    }
    Row row = buildRow(schema, statement);
    table.insert(row);
    executed = true;
    return row;
}
```

最初の next() は挿入した Row を返し、次の next() は null を返します。

UpdateOperator と DeleteOperator

更新や削除を行うためには、データの中身 (Row) だけでなく、「物理的にディスクのどこにあるのか (RecordId)」という情報が必要です。そのため、子オペレータから Row を受け取る通常のイテレータモデルとは異なり、内部で直接 Table.scanRecords() (Row と RecordId のペアを返す) を利用して処理を行います。

▼ 条件に一致した 1 行を更新する処理 (UpdateOperator)

```
while (iterator != null && iterator.hasNext()) {
    Table.Record record = iterator.next();
    Row row = record.row();

    if (ConditionEvaluator.matches(
        row, statement.whereCondition())) {
        Object newValue =
            parseValue(statement.value(), targetColumn);
        row.put(targetColumn.name(), newValue);

        // RecordIdを指定して上書き
        table.update(record.recordId(), row);
        return row;
    }
}

return null;
```

一致する行が複数あれば、next() が呼ばれるたびに 1 行ずつ処理して結果を返します。テーブルオブジェクトへ更新を委譲するため、第7章で実装したインデックスの同期処理もそのまま利用されます。

8.7 プランナによる実行計画ツリーの構築

オペレータという「部品」が揃いました。次はプランナ（Planner）が Statement と Schema からこれらを組み立て、実行計画ツリーを作ります。

単一テーブルの実行計画ツリー

プランナは、最初に木の葉（一番下）となる走査方法を選びます。インデックスを利用できる場合は `IndexScanOperator`、それ以外は `SeqScanOperator` です。

▼ 走査方法を選択する処理

```
Operator plan;

if (condition != null && isIndexable(schema, condition)) {
    plan = new IndexScanOperator(
        table, schema, statement.tableName(), condition);
} else {
    plan = new SeqScanOperator(table, schema, statement.tableName());
}
```

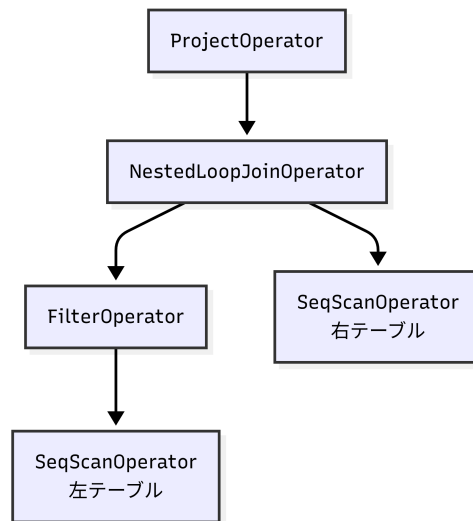
WHERE 条件があればその上に `FilterOperator` を重ね、最後に `ProjectOperator` を重ねます。たとえば `SELECT name FROM users WHERE age >= 20` は、次の実行計画ツリーになります。

```
ProjectOperator [name]
|
FilterOperator [age >= 20]
|
SeqScanOperator [users]
```

プランナがクエリエグゼキュータに返すのは、根（トップ）の `ProjectOperator` だけです。根に `next()` を呼べば自動的に下へ処理が伝わるため、実行側は内部構造を一切気にする必要がありません。

JOIN と条件のプッシュダウン

JOIN では、WHERE 条件を「ツリーのどこに置くか」によってパフォーマンスが激変します。条件が左テーブルだけを参照する場合、`NestedLoopJoinOperator` よりも「下」に `FilterOperator` を置きます。これを条件のプッシュダウンと呼びます。



▲図 8.3

結合（JOIN）の前に左側の行を減らしておくことで、比較する組み合わせの数が激減します。この実装は限定的ですが、オペレータの配置順序（実行計画）を変えるだけで、結果を変えずに処理量を劇的に減らせることが体験できます。

8.8 クエリエグゼキュータによるオペレータの実行

プランナが実行計画ツリーを作り、すべての処理がオペレータに抽象化されたことで、クエリエグゼキュータは SQL ごとの検索方法や処理順序を管理する必要がなくなりました。

CREATE TABLE 以外の Statement はすべてプランナへ渡し、根のオペレータを取得します。

▼ プランナから根のオペレータを受け取る処理

```

public void execute(Statement statement) throws IOException {
    if (statement instanceof Statement.CreateTable createTableStmt) {
        executeCreateTable(createTableStmt);
    } else {
        Operator rootPlan = planner.createPlan(statement);
        executePlan(rootPlan, statement);
    }
}

```

実行時は `open()` を呼び、`next()` が `null` を返すまでループで `Row` を受け取り続けます。最後に必ず `close()` を呼べるように、終了処理を `finally` ブロックへ置きます。

▼ オペレータを実行する共通ループ

```
private void executePlan(
    Operator rootPlan, Statement statement) throws IOException {
    rootPlan.open();

    try {
        int count = 0;
        while (true) {
            Row row = rootPlan.next();
            if (row == null) {
                break;
            }
            if (statement instanceof Statement.Select) {
                System.out.println(row);
            }
            count++;
        }

        // Statementに応じて空の結果や更新件数を表示する
    } finally {
        rootPlan.close();
    }
}
```

SELECT では `Row` を表示し、UPDATE と DELETE では `next()` が返した回数を処理件数として表示します。クエリエグゼキュータが「今何のオペレータを動かしているのか」を一切意識しないことが、イテレータモデルの最大の利点です。

8.9 実行結果の確認

本章の変更はデータベース内部のアーキテクチャを整理するものであり、利用者が入力する SQL やその結果は第 7 章から変わりません。

```
db > SELECT name FROM users WHERE age >= 20;
{name=Taro}

db > UPDATE users SET age = 21 WHERE id = 1;
Updated 1 row(s).

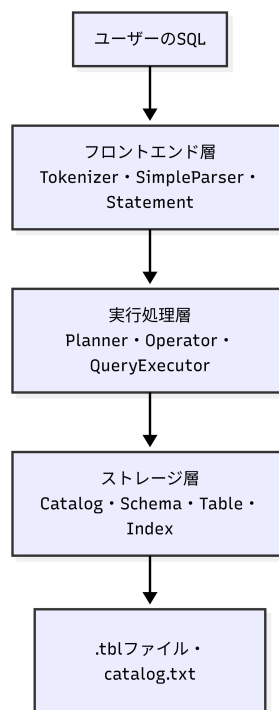
db > DELETE FROM users WHERE id = 2;
Deleted 1 row(s).
```

SELECT は走査、絞り込み、射影をつないだ実行計画ツリーで処理され、UPDATE と DELETE は共通の実行ループで件数がカウントされます。外部のインターフェースを

保ったまま内部構造を劇的に変更できたことは、コンポーネントの責務が美しく分離されている証拠です。

8.10 まとめと次章への課題

第1章では、データベース全体をフロントエンド層、実行処理層、ストレージ層に分けました。本章のオペレータ導入により、実装上のクラスが見事にこの役割へと対応しました。



▲図 8.4

構文解析が SQL を AST へ変換し、プランナがオペレータを組み合わせます。クエリエグゼキュータは実行計画ツリーを共通の方法で動かし、テーブルとインデックスは行の保存と検索に集中します。

一方、本章の冒頭で提起した「メモリ枯渇問題」は、実はまだ完全に解決していません。現在の SeqScanOperator の実装を振り返ると、open() の中で Table.scan() を呼び

出し、その戻り値である巨大な `List<Row>` をイテレータに変換して使用しています。上位のオペレータ間では 1 行ずつ受け渡す美しいバケツリレーが完成しましたが、「一番底のストレージからの読み込み」は一括読み込みのままなのです。

これを完全に 1 行ずつ処理（ストリーミング化）し、真の意味でメモリ消費を抑えるには、Table 側にページとスロットを順に 1 件ずつ読み進める「Cursor（カーソル）」という仕組みを実装する必要があります。

```
現在
Table.scan() ——(一括)——▶ List<Row> ——▶ SeqScanOperator.next()

真のストリーミング化（今後の課題）
Table Cursor ——(1行ずつ)——▶ SeqScanOperator.next()
```

また、本章のプランナが行う最適化は、インデックス走査の選択と限定的なプッシュダウンだけです。実際の商用データベースでは、統計情報を使ったコスト推定（CBO）、JOIN 順序の動的変更、Sort や Aggregate など、さらに多くの複雑な処理が行われています。

本章では、それらを新たなオペレータとして追加し、自由に組み合わせるための「強力な土台」を完成させました。次章（第 9 章）では、最小のデータベースからここまで追加してきた仕組み全体を振り返り、本物の実用的なデータベースシステムに到達するために残された機能と、今後の発展について整理します。

第 9 章

真のデータベースに向けて

これまでの章を通して、私たちは何もない状態から一步步、データベースシステムの核となる仕組みを自作してきました。対話型の小さなプログラムから始まり、ディスクへの永続化、ページベースのストレージ管理、B-Tree による高速な検索、SQL の構文解析からクエリプランナの実装、そして第 8 章でのアーキテクチャの整理まで、データベースの裏側で動いている要素技術を体験しました。

本章では、これまでの歩みを振り返るとともに、私たちが作成したデータベースを実運用に耐える本格的なシステムへと進化させるために「今足りないものは何か」、そして「それを今後どう実装していくのか」という方針について解説します。

9.1 これまでの振り返り

私たちが構築したデータベースは、現在以下のような構成を持っています。

- **ストレージエンジン (第 3 章～第 5 章)**：データをファイルに永続化し、固定長スロットとページという単位でディスクを管理。さらに B-Tree を用いて、線形探索に頼らない高速なインデックスとデータ配置を実現しました。
- **実行エンジン (第 6 章～第 7 章)**：パーサ (構文解析)、プランナ (実行計画の選択)、クエリエグゼキュータ (クエリの実行) を実装し、ユーザーが入力した SQL 文字列から最適な手順を導き出し、結果を返すパイプラインを構築しました。
- **構造の分離と整理 (第 8 章)**：システムが肥大化する中で、ストレージ管理、カタログ (メタデータ) 管理、クエリ実行などの各コンポーネントの責任境界を明確にし、拡張性の高いアーキテクチャへとリファクタリング (オペレータの導入) を行いました。

これにより、単一のアプリケーションからローカルファイルとして直接扱う「組み

込み型データベース（SQLite のような形式）」の基礎は十分に完成しました。しかし、PostgreSQL や MySQL のような堅牢なデータベースサーバになるためには、まだいくつかの大きな壁があります。

9.2 今に足りないものと実装方針

私たちが作ったデータベースをさらに拡張するための、4 つの大きなテーマとその実装方針を紹介します。

クライアント／サーバモデル（ネットワーク対応）

【現状と課題】現在のシステムは、コマンドライン（REPL）上で直接プロセスとして動作しており、他のアプリケーションからネットワーク越しに利用することができません。

【実装方針】データベースを常駐する「サーバプロセス」として独立させます。

- **通信プロトコル:** TCP/IP ソケット通信を用いて、複数クライアントからの接続を同時に受け付けるデーモンプログラムを作成します。
- **ワイヤープロトコル:** クライアントから SQL 文字列を受け取り、実行結果（カラム定義と Row の集合、またはエラー情報）をバイナリ形式でシリアル化して送り返す独自の通信ルール（プロトコル）を設計します。

トランザクションと同時実行制御（Concurrency Control）

【現状と課題】複数のユーザーが同時にデータを読み書きすることを想定していません。同時に更新処理が走ると、データファイルが競合して破損する（不整合が起きる）危険性があります。

【実装方針】ACID 特性（特に分離性：Isolation）を満たすための制御機構を組み込みます。

- **ロック機構の導入:** レコード単位やページ単位で、「共有ロック（Read Lock）」と「排他ロック（Write Lock）」を取得・解放する Lock Manager を実装し、競合を防ぎます。
- **MVCC（多版同時実行制御）:** さらに高度な方針として、データの更新時に上書きせず「新しいバージョンのレコード」を作成することで、読み取り処理と書き込み処理が互いをブロックしない仕組み（PostgreSQL などが採用）の実装に挑戦するのも良いでしょう。

障害復旧とログ (Crash Recovery)

【現状と課題】メモリ上（バッファプール）にある変更がディスクに書き込まれる前にシステムがクラッシュしたり電源が落ちたりすると、データが失われたり、B-Tree の構造が途中で壊れたりします。

【実装方針】トランザクションの永続性 (Durability) と一貫性 (Consistency) を保証するために、ログ先行書き込み (WAL: Write-Ahead Logging) を実装します。

- **WAL ファイルの実装:** データファイル（ページ）を更新する前に、必ずシーケンシャルな「ログファイル」に変更内容を追記 (Append) します。
- **リカバリ機構:** データベースの起動時に WAL を読み込み、クラッシュ前に完了していた処理を再実行 (Redo) し、途中で終わっていた処理を取り消す (Undo) 機構を実装します。

より高度なクエリオプティマイザと SQL サポート

【現状と課題】現在のプランナは、インデックスの有無などの単純なルールベースで動いています。また、集約関数 (COUNT など) や複雑な条件には対応していません。

【実装方針】

- **集約とソート:** GROUP BY や ORDER BY を処理するために、メモリ上でのハッシュ集約や、メモリに乗り切らない場合の外部ソートアルゴリズムを実行エンジンに追加します。
- **コストベース・オプティマイザ (CBO):** カタログに統計情報（テーブルの行数や、カラムの値の分布など）を保持させます。その情報をもとに、「フルスキャンとインデックススキャンのどちらが I/O コストが低いか」を数学的に計算して最適な実行計画を選択するようにプランナを拡張します。

9.3 おわりに：ブラックボックスを開けよう

本書の目的は、単に動くツールを作ることではなく、「データベースという巨大なブラックボックスの蓋を開け、自分の手で仕組みを理解すること」でした。

普段何気なく使っている `SELECT * FROM users WHERE id = 1;` というたった一行の裏側で、文字列がトークンに分解され、構文木が構築され、プランナが最適な道を選び、B-Tree のルートノードからリーフへと辿り、ページからスロットを読み取って結果を返すという、壮大なバケツリレーが行われています。

一度でも自らの手でデータベースを作り上げ、内部の構造を分離し整理したあなたは、今後 PostgreSQL や MySQL を使う際、システムを単なる「ブラックボックス」ではなく、「アルゴリズムとデータ構造の結晶」として捉えられるようになっているはずです。

データベースの世界は深く、そして広大です。本書で築いた「最小のデータベース」を足がかりに、ぜひご自身の手で足りない機能を実装し、さらに高度なデータ基盤の世界へと足を踏み入れてみてください。

あとがき

最後まで本書をお読みいただき、本当にありがとうございました。

「データベースをゼロから自作する」という本書の試みは、いかがだったでしょうか。おそらく、第5章の「B-Treeのノード分割」や、第6章の「構文解析（パーサ）の実装」あたりで、頭を抱えながらコードを追った方も少なくないと思います。決して平坦な道のりではありませんでしたが、その壁を乗り越えて実行計画が動き、高速にデータが返ってきたときの「カタルシス」を少しでも味わっていただけたなら、筆者としてこれ以上の喜びはありません。

私たちが本書を執筆しようと思いついたのは、私たち自身が長らく「データベースのブラックボックス化」に悩まされていたからです。アプリケーションエンジニアとして日々SQLを書き、インデックスを張ってチューニングをしているつもりでも、心のどこかで「本当は裏側で何が起きているのか分かっていない」というもどかしさがありました。世の中には優れたデータベースの理論書がたくさんありますが、数式や概念だけでなく「実際に手を動かして、課題にぶつかりながら学ぶ」アプローチの本があれば良いのに――そう考えたのが、本書の出発点です。

本書を通じて皆さんと一緒に組み立てたのは、「最小のデータベース」です。トランザクションもなければ、ネットワーク越しに接続することもできません。しかし、ここで書いた数千行のコードの延長線上に、私たちが普段使っている PostgreSQL や MySQL といった巨大なエコシステムが存在しています。本書を読み終えた皆さんは、もうデータベースをただの「魔法の箱」とは思わないはずです。今後、実務で重いクエリに直面したとき、頭の中に「実行計画ツリー」や「B-Treeのノード」が自然と思い浮かぶようになっていれば幸いです。

データベースの世界は、知れば知るほど奥深く、そして面白いです。本書が、皆さんのエンジニアリング人生において、さらに深い技術の海へ飛び込むための「最初の羅針盤」となることを願っています。

2026年7月くろねこチーム

著者紹介

久保 友樹

趣味：バイク（FTR、R1-Z（予定））

小林 龍生

github: ryu-k67

佐佐木 悠人

github: Yutosaki Student at 42Tokyo

白根 薫

github: RuoHacker

0 から作る自作 DB 入門

2026 年 7 月 12 日 v1.0.0

著 者 久保 友樹、小林 龍生、佐佐木 悠人、\白根 薫

編 集 mhidaka

発行所 くろねこチーム

(C) 2026-2035 くろねこチーム

